



POLITECHNIKA KRAKOWSKA im. T. Kościuszki
Wydział Inżynierii Elektrycznej i Komputerowej
Katedra Automatyki i Informatyki E-1



Kierunek studiów: Informatyka w inżynierii komputerowej

Specjalność: —

STUDIA STACJONARNE

PRACA DYPLOMOWA

INŻYNIERSKA

Paweł SOKÓŁ

PROJEKT I IMPLEMENTACJA POLIFONICZNEGO SYNTEZATORA
DŹWIĘKOWEGO Z WYKORZYSTANIEM SYSTEMU
OPERACYJNEGO CZASU RZECZYWISTEGO

DESIGN AND IMPLEMENTATION OF A POLYPHONIC SOUND
SYNTHESIZER USING A REAL-TIME OPERATING SYSTEM

Opiekun pracy:
dr inż. Paweł Król

Kraków, 2025

Spis treści

Streszczenie w języku polskim i angielskim	3
Wykaz słów kluczowych w języku polskim i angielskim	4
1. WSTĘP	5
1.1. Wprowadzenie.....	5
1.2. Cel i zakres pracy	6
1.3. Przegląd zawartości pracy	6
2. PODSTAWY TEORETYCZNE ZWIĄZANE Z TEMATEM PRACY	7
2.1. Co to jest dźwięk?	7
2.2. Barwa dźwięku, alikwoty, analiza fourierowska	8
2.3. Dyskretyzacja sygnału dźwiękowego	11
2.3.1. Aliasing	12
2.3.2. Szum kwantyzacji	14
2.4. Podstawy działania syntezy dźwiękowego	15
2.4.1. Rodzaje syntezy dźwięku.....	16
2.4.1.1. Generator tonów prostych i szumów	16
2.4.1.2. Synteza tablicowa	16
2.4.1.3. Synteza samplingowa	16
2.4.1.4. Synteza FM i AM	16
2.4.2. Budowa oscylatora.....	17
2.4.3. Rodzaje oscylatorów	17
2.4.4. Modularność	18
2.4.5. Opis pracy syntezy	21
2.4.6. Polifoniczność syntezy	22
3. OPIS REALIZACJI TEMATU PRACY	23
3.1. Studium wykonalności projektu.....	23
3.1.1. Program testowy napisany w języku C/C++	23
3.1.2. Program testowy napisany w języku assembler	26
3.2. Projekt, założenia projektowe i architektura systemu.....	31
3.3. Implementacja	32
3.3.1. Składowanie i organizacja danych.....	32
3.3.2. Implementacja oscylatorów	35
3.3.3. Opis pracy przerywania	36
3.3.4. Interfejs programistyczny	44
3.3.5. Funkcje pomocnicze	45
3.3.6. Rozszerzenie projektu o system RTOS	46
3.3.6.1. Implementacja biblioteki FreeRTOS	47
3.3.6.2. Analiza rezultatów implementacji FreeRTOS	49
3.3.6.3. Założenia projektowe dedykowanego systemu RTOS	50
4. ANALIZA WYNIKÓW	52
5. PODSUMOWANIE.....	53
6. BIBLIOGRAFIA	54

Streszczenie w języku polskim i angielskim

Cel projektu jaki należało zrealizować to stworzenie wirtualnego syntezyzatora dźwiękowego w postaci biblioteki na mikrokontrolerze AVR ATmega328P. Wybór padł na taki mikrokontroler ponieważ jest kompatybilny z popularną platformą arduino. Stworzenie takiego syntezyzatora dźwiękowego umożliwiło łatwą generację melodii i odtwarzanie bardziej złożonych kompozycji muzycznych. To zadanie było szczególnie trudne do zrealizowania ze względu na ograniczoną moc obliczeniową tego mikrokontrolera, a także ubogie zasoby dostępnej pamięci operacyjnej i programowej. W tej pracy należało wykazać się więc dużą sprawnością programistyczną aby skutecznie zoptymalizować kod pod architekturę tego mikrokontrolera, tak aby umożliwić odtworzenie muzyki polifonicznej. W tym celu zostały zastosowane nietypowe i niestandardowe rozwiązania takie jak łączenie kodu assemblera z kodem C++ i inne. W tej pracy zawarto szczegółowy opis decyzji i rozwiązań projektowych, proces implementacji, a także podstawę teoretyczną która jest niezbędna do zrozumienia zasady działania i funkcjonowania aplikacji.

The goal of the project that had to be realized was to create a virtual sound synthesizer in the form of a library on the AVR ATmega328P microcontroller. Such a microcontroller was chosen because it is compatible with the popular Arduino platform. Creating such a sound synthesizer allowed easy generation of melodies and playing more complex musical compositions. This task was particularly difficult to accomplish due to the limited computing power of this microcontroller, as well as the poor resources of available RAM and program memory. In this work, it was necessary to demonstrate great programming skills in order to effectively optimize the code for the architecture of this microcontroller, so as to enable the playback of polyphonic music. For this purpose, unusual and non-standard solutions were used, such as combining assembler code with C++ code and others. This work contains a detailed description of design decisions and solutions, the implementation process, as well as the theoretical basis that is necessary to understand the principle of operation and functioning of the application.

Wykaz słów kluczowych w języku polskim i angielskim

Dźwięk, ton, nuta, nuty – są to pojęcia bliskoznacznie i często wymiennie stosowane. Dźwięk to wrażenia słuchowe, bodziec jakie wywołują fale mechaniczne u odbiorcy. Tonem określane jest często wysokość dźwięku, a nuta to symbol używany w muzyce do zapisania dźwięku instrumentu o określonej wysokości i czasie trwania. Przyjęło się także określenie ton prosty jako sygnał okresowy o przebiegu sinusoidalnym, a także ton podstawowy jako sygnał okresowy pozbawiony składowych harmonicznych.

Polifonia – w kontekście syntezatorów dźwiękowych oznacza to zdolność instrumentu do odtwarzania wielu dźwięków jednocześnie. Liczba głosów to jednostka określająca zdolność syntezatora do tego, ile jest w stanie tych dźwięków wygenerować jednocześnie.

Monofoniczność, stereofoniczność – to określenie na ilość ścieżek audio. Stereofonia oznacza, że tych ścieżek są odpowiednio dwie: dla lewego jak i prawego ucha, co zapewnia przestrzenność dźwięku. Monofonia to określenie na jedną ścieżkę audio w którym nie jest możliwe odwzorowanie przestrzenności dźwięku. Ścieżka audio jest często w tym przypadku nazywana jako kanał np. kanał lewy, kanał prawy.

Przebieg czasowy, sygnał okresowy – jest to przedstawienie wykresu funkcji sygnału w zależności od czasu. Sygnał okresowy to szczególny przypadek przebiegu czasowego w którym zachodzi powtarzalność tego sygnału.

Próbkowanie, częstotliwość próbkowania – to sposób zapisu przebiegu czasowego w którym pobierana i zapisywana jest wartość chwilowa sygnału w określonych momentach czasu. Częstotliwość próbkowania to ilość pobieranych próbek dźwiękowych z sygnału na sekundę.

Oscylator – jest to moduł generujący proste sygnały okresowe lub szum

Modulacja, modulowanie, modularność – modulowanie to proces manipulacji parametrami oscylatorów. Można wyróżnić w tym przypadku wyróżnić modulację amplitudową (AM) lub częstotliwościową (FM). Modularność chodź jest to podobnie brzmiące słowo to pochodzi od słowa moduł. Oznacza to możliwość dowolnego łączenia modułów w większe, bardziej złożone struktury.

1. WSTĘP

1.1.Wprowadzenie

Arduino to popularna platforma do tworzenia elektronicznych projektów i prototypów, a także do nauki języka programowania. Na uwagę zasługuje prostota środowiska programistycznego która zachęca nowych użytkowników do pracowania na tej platformie, co w znaczącym stopniu zwiększa jej popularność. Jednym z wielu wykorzystywanych mikrokontrolerów są te oparte na architekturze AVR opracowanej przez firmę Atmel, a w szczególności rozpowszechnionym mikrokontrolerem jest ATmega328P. To właśnie na tym mikrokontrolerze zostanie zrealizowany ten projekt tj. syntezy dźwiękowy.

System wirtualnego syntezy dźwięku dołączany za pomocą biblioteki zapewni łatwość i prostotę implementacji udźwiękowienia w projektach które mogą tego wymagać jak np. w retro-grach typu brick games, a także innych podobnych rozwiązaniach. Może się też sprawdzić przy generowaniu prostych sygnałów okresowych i różnych przebiegów czasowych mogących przetestować działanie innych układów elektronicznych. Jednak głównym zastosowaniem tej biblioteki pozostanie nadal odtwarzanie muzyki w tle, nie zakłócając działania głównej aplikacji i stworzenie systemu „all in one chip”, zmniejszając koszty implementacji takiego rozwiązania dla innych potencjalnych użytkowników tego systemu.

Podsumowując zaletą wyboru tego mikrokontrolera jest jego popularność. Wadą są natomiast ograniczenia sprzętowe wynikające głównie z ubogiej w zasoby pamięci i mocy obliczeniowej tego mikrokontrolera.

1.2. Cel i zakres pracy

Celem tego projektu jest opracowanie rozwiązania zapewniającego możliwość odtwarzania dźwięku na mikrokontrolerze AVR ATmega328P w tle tzn. niezależnie od działania głównej aplikacji. Rozwiązanie będzie dołączane do głównego kodu programu w postaci biblioteki. Takie podejście zapewni innym potencjalnym użytkownikom łatwość implementacji kodu nie rezygnując przy tym z części swojej funkcjonalności.

Zakres pracy obejmuje stworzenie takiej biblioteki która będzie miała możliwość:

- Jednoczesnej symulacji pracy wielu oscylatorów które są niezbędne w procesie syntezy dźwięku – zrealizowanej przy użyciu przerwania aby zapewnić stałą częstotliwość próbkowania dźwięku, a także dużą częstotliwość pracy
- Implementacja systemu RTOS do wykonywania wielu bardziej zaawansowanych zadań jednocześnie, ale nie wymagających dużej częstotliwości pracy
- Możliwość tworzenia swoich własnych konfiguracji brzmieniowych dźwięków
- Prosty interfejs programistyczny który pomoże w zarządzaniu pracą syntezy
- Optymalizacja kodu aby zmniejszyć zużycie zasobów procesora i pamięci do minimum przyspieszając w ten sposób działanie aplikacji
- Stworzenie programu który przetestuje pracę tej biblioteki

1.3. Przegląd zawartości pracy

W pracy zostaną zawarte teoretyczne podstawy które są niezbędne do pełnego zrozumienia zasady działania aplikacji.

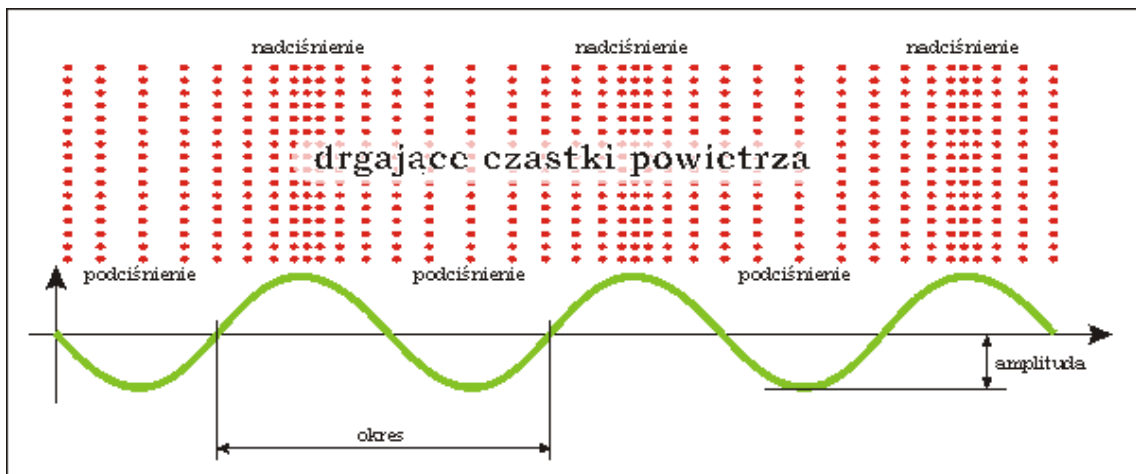
Następnie zostanie omówiony proces implementacyjny który ukazuje problemy z jakimi należało się zmierzyć podczas tworzenia tej aplikacji. Każda próba implementacji daje kluczowe znaczenie dla podjętych decyzji projektowych. Zostaną także omówione różne sposoby rozwiązania napotkanych problemów i to, które rozwiązania dadzą najlepsze korzyści w konkretnym zastosowaniu.

Na zakończenie zostaną pokazane rezultaty osiągniętych prac, a także zostaną zaproponowane dalsze możliwości rozwinięcia tego projektu.

2. PODSTAWY TEORETYCZNE ZWIĄZANE Z TEMATEM PRACY

2.1. Co to jest dźwięk?

Dźwięk jest to fala mechaniczna rozchodząca się w danym ośrodku np. w powietrzu która jest zdolna do wywołania wrażeń słuchowych u człowieka lub zwierząt. Fala ta pochodzi od drgających przedmiotów np. instrumentów muzycznych, które drgając emitują te fale mechaniczne trafiające następnie do słuchaczy. Fale te można opisać za pomocą funkcji zmiany ciśnienia powietrza (lub innego ośrodka drgającego) w zależności od czasu tak jak to przedstawiono na rys. 1.



Rysunek 1: Wykres ilustrujący dźwięk jako zmiany w ciśnieniu powietrza w zależności od czasu (źródło: [1])

Fala dźwiękowa może przyjmować różne właściwości, w zależności od tego z jakiego źródła pochodzi. Dzięki temu możliwe jest rozróżnianie dźwięków, ich pochodzenia, a także możliwa jest komunikacja werbalna między dwoma osobami. Do podstawowych własności dźwięku należą [2]:

- Częstotliwość – określa z jaką szybkością drga dany przedmiot, przyjęło się umownie że człowiek słyszy dźwięki z zakresu od 20Hz do 20kHz
- Amplituda, lub inaczej głośność – jak duże są drgania danego przedmiotu
- Barwa dźwięku – inaczej kształt fali dźwięku. Każdy rodzaj fali dźwięku jest typowy dla ośrodka który go wygenerował. W ten sposób możliwe jest rozpoznawanie przedmiotu emitującego dźwięk, a także również kto jest mówcą w rozmowie. To zagadnienie zostanie szerzej opisane w kolejnym dziale 2.2.

Każdy z powyżej wymienionych parametrów również podlega zmianą w czasie. Te zmiany również są kluczowe podczas identyfikacji dźwięków przez człowieka. Np. fortepian po naciśnięciu klawisza instrumentu wyda dźwięk, jednak z czasem

drgająca struna traci swoją energię zmniejszając tym samym swoją głośność w czasie. Zmiana głośności w czasie jest znana pod pojęciem obwiedni.

2.2. Barwa dźwięku, alikwoty, analiza fourierowska

O ile dźwięk drgającego przedmiotu takiego jak np. instrument muzyczny da się przedstawić jako przebieg który okresowo się powtarza, tak można zauważyć że taki przebieg będzie często dość „nieregularny” w swoim okresie. Ta właśnie nieregularność odpowiada za barwę dźwięku, jednak reprezentacja jej w takiej postaci jest mało czytelna. Trudno w niej powiedzieć jak zmiana kształtu fali będzie mieć wpływ na barwę dźwięku. Trudno jest także porównywać takie fale między sobą, ponieważ może się okazać że dwa różne kształty fal w swoim przebiegu będą brzmieć praktycznie identycznie.

Tak więc do opisu barwy dźwięku dużo lepszym wyborem jest ich „rozbicie” na jednakowe i dużo prostsze kształty fal o przebiegu sinusoidalnym. W ten sposób możliwe jest łatwe zauważenie co w takiej fali się znajduje i co się zmienia, a także które składowe fali mają swój największy udział w odbiorze dźwięku. W tym celu zostanie wykorzystany dział analizy matematycznej o nazwie analiza fourierowska. Analiza ta polega właśnie na podziale danej funkcji w określonym zakresie na sumę funkcji sinusoidalnych i cosinusoidalnych według poniższego wzoru Fouriera:

Niech funkcja $f(x)$ będzie funkcją okresową o okresie T . Do funkcji tej jest zbieżny szereg o postaci [3]:

$$f(x) = \frac{a_0}{2} + \sum_{k=1}^{\infty} [a_k \sin(kx) + b_k \cos(kx)] \quad (1)$$

gdzie:

$$a_0 = \frac{2}{T} \int_0^T 1 \cdot f(x) dx$$

$$a_k = \frac{2}{T} \int_0^T \sin(kx) f(x) dx$$

$$b_k = \frac{2}{T} \int_0^T \cos(kx) f(x) dx$$

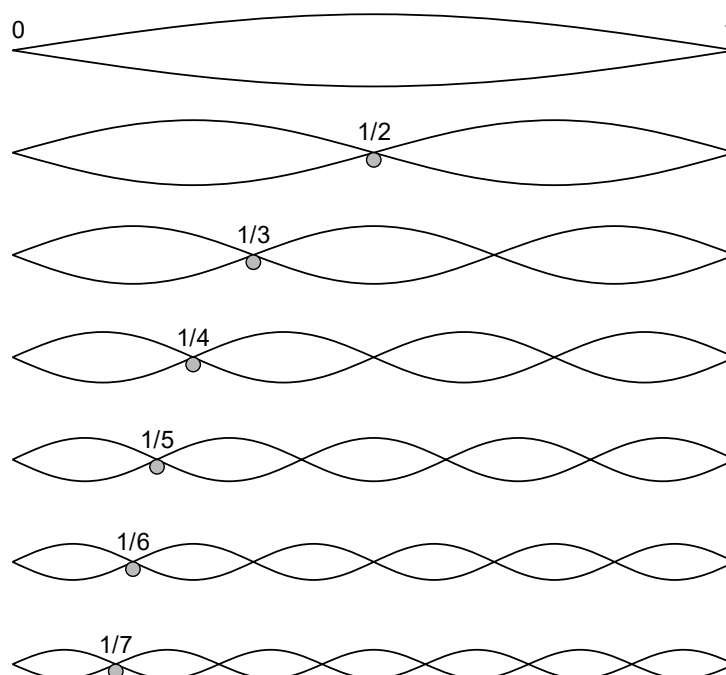
Ponieważ $\cos$ to $\sin$ przesunięty o 90° to możliwe jest przeliczenie powyższego wzoru do postaci takiej, w której:

$$a \cdot \sin(x) + b \cdot \cos(x) = c \cdot \sin(x + \alpha)$$

Można wtedy zauważyć że za pomocą funkcji $\sin$ i $\cos$ w powyższym wzorze opisywane jest tak naprawdę przesunięcie fazowe funkcji sinusoidalnej względem innych składowych harmoniczych. Z tego więc wynika dlaczego dwa różne wykresy przebiegu okresowego dźwięku mogą brzmieć tak samo – składowe harmoniczne mogą być przesunięte fazowo między sobą, ale częstotliwości składowe i ich amplitudy pozostają niezmiennie.

Podsumowując, właściwości barwy dźwięku można opisywać za pomocą tonu podstawowego o przebiegu sinusoidalnym, a także składowych harmoniczych które są wielokrotnością częstotliwości tonu podstawowego. Potocznie szereg harmoniczny jest nazywany także jako szereg alikwotów [4].

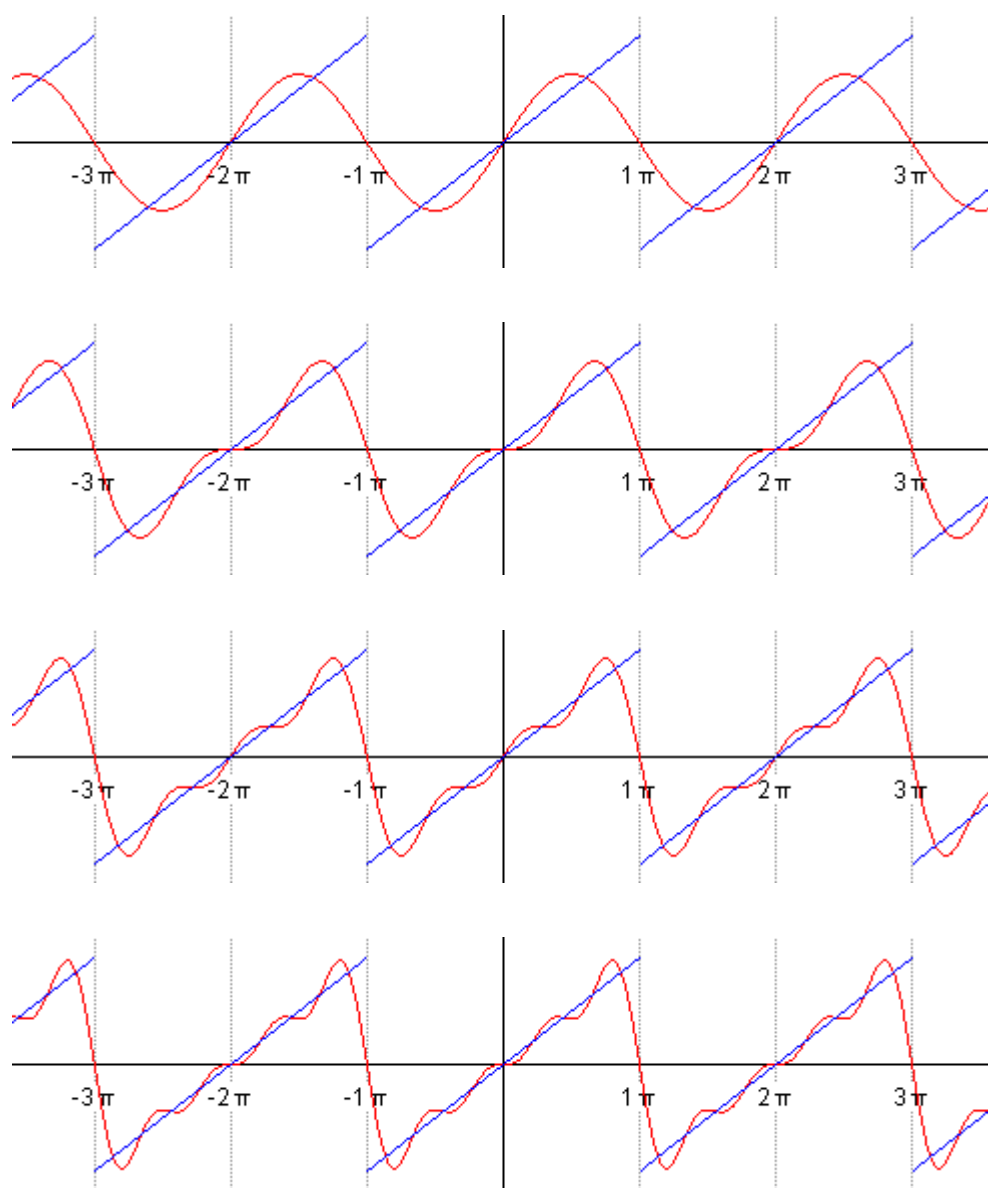
Pojęcie alikwoty znane już było w starożytnej Grecji (ok. 500 p.n.e.) i zostało odkryte przez Pitagorasa. Już wtedy zauważono że drgająca struna instrumentu może „wychylać się” na całej swojej długości, lub dzielić się na równe części [5] tak jak na poniższym rysunku.

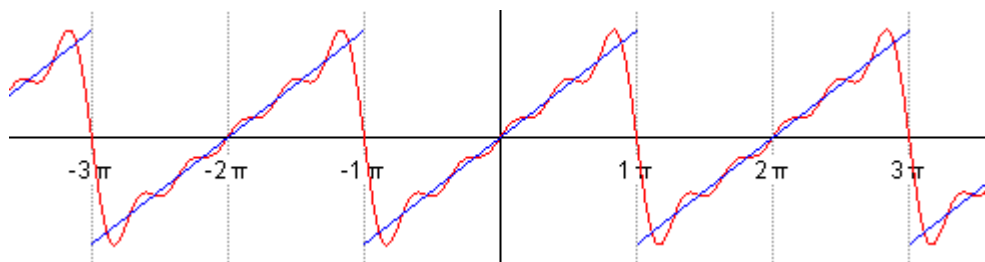


Rysunek 2: Przedstawienie w jaki sposób struna może „podzielić” swoje drgania tworząc alikwoty (źródło: [5])

To bardzo dobrze ilustruje jak istotne znaczenie mają dla barwy dźwięku składowe harmoniczne. Ponieważ dźwięki pochodzenia naturalnego mają tendencje do wytwarzania składowych harmonicznych, to sposób wydobywania takiego dźwięku determinuje jaka zostanie uzyskana struktura amplitud składowych harmonicznych. Co za tym idzie możliwe jest określenie barwy dźwięku, a także jego źródła pochodzenia i to jak został on wydobyty.

Przykład kolejnych etapów powstawania szeregu harmonicznego na podstawie syntetycznej fali piłokształtnej. W każdym kolejnym etapie dodawana jest kolejna składowa harmoniczna będąca wielokrotnością tonu podstawowego.



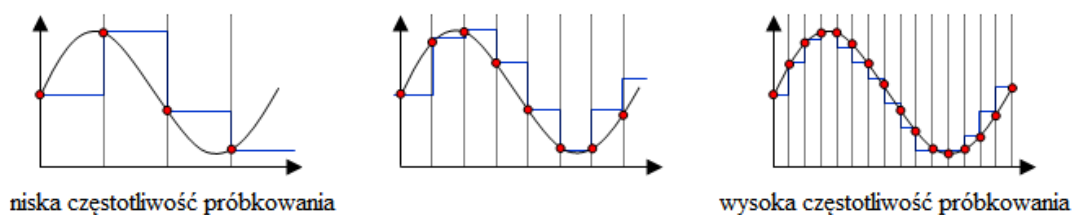


Rysunek 3: Przedstawienie pięciu kolejnych wyrazów szeregu Fouriera aproksymującego funkcję pilokształtną (źródło: [6])

2.3. Dyskretyzacja sygnału dźwiękowego

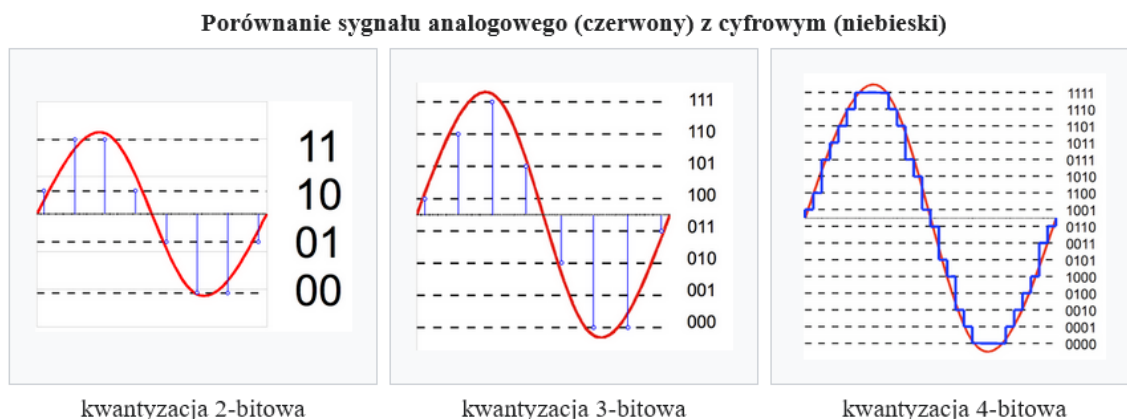
Dźwięki występujące w naturze mają ciągły charakter drgań (lub inaczej liniowy). Problemem może okazać się więc próba odwzorowania takiego sygnału ciągłego przez układy cyfrowe. Budowa takiego układu uniemożliwia jego wierne odtworzenie ponieważ takie układy działają w oparciu o założenia matematyki dyskretnej, to znaczy obliczenia korzystają wyłącznie tylko z zer i jedynek. Takie układy mają więc skończoną dokładność ponieważ operują na skończonej liczbie wartości dyskretnych. Również takie układy muszą prowadzić stale obliczenia które zajmują pewien okres czasu. Z tego powodu nie jest możliwe dokładne odwzorowanie oryginalnego sygnału analogowego, ale za to możliwe jest odwzorowanie jego przybliżonej reprezentacji [7]. Dokładność takiego odwzorowania będzie zależeć od dwóch następujących parametrów:

- Częstotliwości próbkowania – czyli jaka jest dokładność w czasie emitowanych, lub rejestrowanych wartości chwilowych sygnału dźwiękowego [8]



Rysunek 4: Przedstawienie wpływu częstotliwości próbkowania na jakość odtwarzanego sygnału (źródło: [9])

- Głębokość bitowa – czyli z jaką dokładnością są emitowane lub rejestrowane wartości chwilowe sygnału dźwiękowego [10]

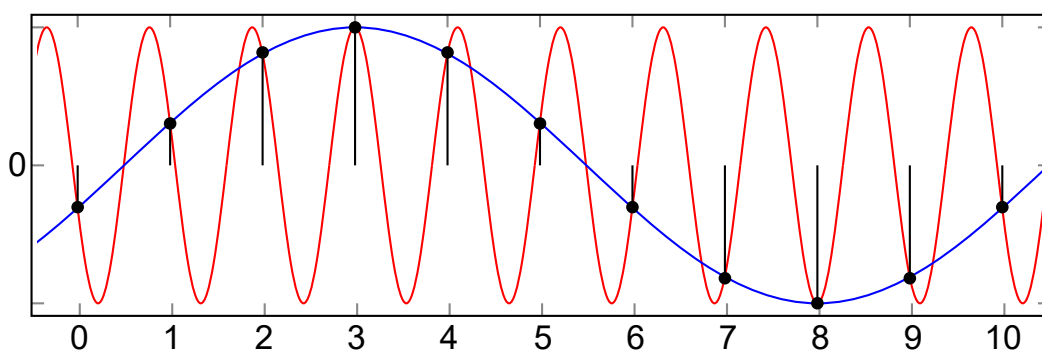


Rysunek 5: Przedstawienie wpływu głębokości bitowej na dokładność odtwarzanego sygnału (źródło: [11])

Takie rozwiązanie, czyli wykorzystanie układów cyfrowych niesie ze sobą wiele wad i zalet. Główną zaletą jest stała dokładność i powtarzalność emitowanego sygnału. Parametry układów analogowych nie są zawsze identyczne i mogą się delikatnie różnić. Również układy analogowe są wrażliwe na czynniki środowiskowe takie jak np. temperatura więc mogą się rozstrajać. Dodatkowo układy cyfrowe są generalnie tańsze, a także łatwiejsze w rekonfiguracji niż układy analogowe. Wadą natomiast układów cyfrowych jest ich skończona dokładność co w rezultacie może prowadzić do niekorzystnych zjawisk zwanych jako aliasing lub szum kwantyzacji.

2.3.1. Aliasing

Aliasing to zniekształcenie sygnału wynikające ze zbyt niskiej częstotliwości próbkowania. Aby możliwe było odtworzenie sygnału okresowego muszą wystąpić co najmniej dwie próbki danych na jeden okres fali. W innym przypadku nie będzie możliwe odtworzenie oryginalnej fali, a zamiast niej może się pojawić fala o innej częstotliwości.



Rysunek 4: Przykład dwóch różnych sinusoid pasujących do tego samego wzoru próbek (źródło: [12])

Na rysunku 6 można zauważyć jak oryginalna fala w kolorze czerwonym nie jest odwzorowywana ze względu na zbyt małe próbkowanie. W rezultacie zostanie więc zarejestrowana i odebrana niebieska fala o dużo niższej częstotliwości. Możliwe jest nawet obliczenie jaka zostanie otrzymana częstotliwość gdy to założenie nie zostanie spełnione według poniższego wzoru [12]:

$$\text{Dla } f_{in} \in \left(-\frac{f_s}{2} + k \cdot f_s ; \frac{f_s}{2} + k \cdot f_s \right) :$$

$$f_{out} = f_{in} - k \cdot f_s \quad (2)$$

gdzie:

$$k \in \mathbb{Z}$$

f_s – częstotliwość próbkowania

f_{in} – częstotliwość wejściowa

f_{out} – częstotliwość jaka zostanie odwzorowana

Warto zaznaczyć że częstotliwość o znaku ujemnym odpowiada takiej samej wartości częstotliwości co o znaku dodatnim. Jednak sygnał okresowy o częstotliwości ujemnej nie będzie mieć takiego samego przebiegu jak o częstotliwości dodatniej.

Jak można zauważyć, realnie odtwarzana częstotliwość będzie zawsze co najmniej 2-krotnie mniejsza od częstotliwości próbkowania i tylko taka zostanie zarejestrowana lub wyemitowana przez układ cyfrowy.

Przykład 1:

Sygnał okresowy o kształcie kwadratowym jest odtwarzany z częstotliwością 16,52kHz. Przyjęta częstotliwość próbkowania to 50kHz. Sama częstotliwość bazowa nie przekracza połowy częstotliwości próbkowania więc ta częstotliwość zostanie zachowana. Jednak zakładając że sygnał nie jest w żaden sposób odfiltrowywany to ten sygnał zawiera także inne składowe częstotliwości. Zgodnie z analizą Fourierowską częstotliwość pierwszej składowej będzie 3-krotnością częstotliwości bazowej, a więc wyniesie $16,52 \cdot 3 = 49,56\text{kHz}$. Jest to wartość przekraczająca połowę częstotliwości próbkowania i nie jest możliwe odwzorowanie tego sygnału. Podstawiając te wartości do wzoru tj. $f_s = 50\text{kHz}$, $f_{in} = 49,56\text{kHz}$ można wyliczyć $f_{out} = 49560 - 50000 = -440\text{Hz}$. Jako że częstotliwości o znaku ujemnym odpowiadają częstotliwością o znaku dodatnim, to w rezultacie będzie można bardzo wyraźnie usłyszeć dźwięk „sinusoidalny” o częstotliwości 440Hz i jest to sygnał aliasowy.

W realnym świecie podczas rejestracji dźwięku przez układ cyfrowy jest on najprawdopodobniej wcześniej odpowiednio odfiltrowany, nawet przez mikrofon który może nie być fizycznie zdolny do zebrania częstotliwości wykraczających poza połowę częstotliwości próbkowania. Ale ten problem nie dotyczy tylko sygnałów rejestrowanych fizycznie, a może się pojawić także przy syntetycznie generowanych źródłach. Bez odpowiedniego przygotowania stwarza to realne zakłócenia w sygnale które nie powinny występować. Najprościej jest zwiększyć częstotliwość próbkowania co złagodzi problem, albo można spróbować odfiltrować ten sygnał co może być nieco bardziej złożonym zadaniem, zwłaszcza gdy np. nagranie zostało już źle zrealizowane i zawiera już aliasing.

Podsumowując, częstotliwość próbkowania powinna być co najmniej 2-krotnością najwyższego emitowanego sygnału, ale warto wykorzystać większą ilość próbek. Nawet gdy zjawisko aliasingu nie wystąpi, to może się zdarzyć sytuacja w której sygnał okresowy po prostu zaniknie. Próbkę sygnału może zostać pobierana w takich miejscach w których ten sygnał okresowy przechodzi przez miejsca zerowe. Warto więc ustawić częstotliwość próbkowania na 4-krotność najwyższego sygnału, a nawet na jeszcze większą jeśli jest taka możliwość.

2.3.2. Szum kwantyzacji

Innym zjawiskiem zniekształcającym przebieg czasowy to szum kwantyzacji zwany również jako błędem zaokrąglenia kwantyzacji. Podczas konwersji ciągłego sygnału analogowego na postać cyfrową, każda wartość sygnału źródłowego jest przybliżana do najbliższej wartości dyskretnej [13]. Ten problem także występuje przy syntetycznie generowanych sygnałach, gdzie sygnał przyjmuje wartości pośrednie pomiędzy wartościami dyskretnymi. Zakładając, że nie wykorzystano żadnych technik poprawiających jakość dźwięku jak np. dithering, to takie przybliżanie do najbliższych wartości dyskretnych będzie generować szum w sygnale. W porównaniu do zakłóceń spowodowanych aliasingiem, ten szum będzie przynajmniej na dużo niższym poziomie, ale jest on nadal istotny w kontekście przetwarzania sygnałów dźwiękowych. Korzystając ze wzoru można łatwo policzyć na jakim poziomie głośności taki szum będzie się utrzymywać względem sygnału wykorzystujący cały zakres dostępnych wartości dyskretnych:

$$\text{Szum kwantyzacji} = 10 \cdot \log_{10} \left(\frac{0,5}{2^Q - 1} \right) \text{ dB} \approx -3,01 \cdot (Q + 1) \text{ dB} \quad (3)$$

gdzie:

- Q oznacza ilość bitów rozdzielczości,
- wynik podawany jest w decybelach,

- ilość wartości dyskretnych powinna być nieparzysta aby przebieg czasowy zawierał taki sam zakres dla liczb dodatnich i ujemnych, a także wartość zero

Również to oznacza że poniżej tej wartości nie będzie możliwe odtworzenie sygnałów o niższej amplitudzie. Kolejną kwestią jest też, że amplitudy nagrań dźwiękowych rzadko kiedy wykorzystują w pełni dostępny zakres wartości dyskretnych. W rezultacie udział szumu kwantyzacji będzie realnie większy, zwłaszcza dla cichych fragmentów nagrań, lub takich które wykorzystują wiele ścieżek audio i są następnie miksowane (sumowane ze sobą).

Przykład 2:

W synteźatorze generowany jest dźwięk z dokładnością 8-bitową. Zakładając, że amplituda sygnału dźwiękowego wypełnia cały dostępny zakres wartości dyskretnych, to szum kwantyzacji wyniesie $\approx -27\text{dB}$ względem odtwarzanego sygnału. Natomiast gdy w przebiegu czasowym ma zostać wygenerowanych jednocześnie 16 różnych sygnałów o jednakowej amplitudzie, które następnie zostaną zsumowane, to na każdy sygnał może przypadać tylko 4 bity dokładności (16-krotnie mniej). Brak zmniejszenia zakresu tych amplitud spowodowałby podczas procesu sumowania przepełnienie dostępnego zakresu wartości powodując tak zwane przesterowanie sygnału. Następnie podczas odtwarzania utworu muzycznego, 15 z 16 sygnałów dźwiękowych zakończy swoje odtwarzanie, a pozostanie tylko jeden sygnał ze swoją stałą amplitudą. W rezultacie ilość dostępnych bitów opisujących przebieg czasowy będzie wynosić tylko tyle, ile ten pozostały sygnał. Nie będzie więc wynosić 8 bitów, a tylko 4. Udział szumu kwantyzacji będzie mieć w takim przypadku wartość na poziomie $\approx -15\text{dB}$.

2.4. Podstawy działania synteźatora dźwiękowego

Synteza dźwięku to proces wytwarzania dźwięku w sposób sztuczny, czyli syntetyczny [14]. Dźwięki pochodzenia naturalnego są trudne do odwzorowania w pełni za pomocą algorytmów i wzorów matematycznych, ale możliwe jest odwzorowanie przybliżonej wersji takiego dźwięku. Dodatkowo cyfrowe techniki wytwarzania dźwięku pozwalają na nowe, syntetyczne brzmienia które nie są możliwe do osiągnięcia w naturalny sposób. Każdy typ syntezy dźwięku ma swój unikalny charakter brzmienia co pozwala na tworzenie bardziej urozmaiconych i zróżnicowanych kompozycji muzycznych [15] [16].

2.4.1. Rodzaje syntez dźwięku

2.4.1.1. Generator tonów prostych i szumów

Najprostszym i najbardziej podstawowym rodzajem syntezy dźwięku są generatory tonów prostych i szumów. Generatory, lub inaczej oscylatory wytwarzają w swoim przebiegu czasowym sygnały okresowe o określonym kształcie (wyglądzie), takie jak fala sinusoidalna, kwadratowa, piłokształtna i trójkątna. Generatory mogą wytwarzać także sygnały nieokresowe w postaci szumów. W odbiorze takie dźwięki brzmią typowo syntetycznie.

2.4.1.2. Synteza tablicowa

Bardziej zaawansowaną techniką to synteza tablicowa. Ta technika pozwala na dostosowanie kształtu fali za pomocą tablicy, czyli tak zwanego zbioru próbek dźwiękowych. Cały taki zbiór odpowiada za zapis całego okresu fali dźwiękowej. Możliwe jest tworzenie brzmień, które chociaż częściowo naśladują brzmienie instrumentów muzycznych.

2.4.1.3. Synteza samplingowa

Kolejną techniką to synteza samplingowa. Polega ona na zebraniu próbek dźwiękowych z pojedynczych dźwięków instrumentu w całym jego przebiegu, a nie tylko jego fragmentu tak jak to było w przypadku syntezy tablicowej, tworząc tak zwane „sample”. Ten sposób wymaga dużych zasobów pamięci aby było możliwe zapisanie takiego nagrania, często dość długiego. W rezultacie można spodziewać się całkiem niezłej jakości odwzorowania brzmień instrumentów, jednak dla doświadczonych muzyków może brzmieć dość powtarzalnie.

2.4.1.4. Synteza FM i AM

Kolejną zaawansowaną techniką jest synteza FM lub i AM. Synteza ta polega na modulowaniu jednego z parametrów dźwięku jak np. częstotliwość lub amplituda przez inny oscylator. Oscylatory te działają zazwyczaj z częstotliwością słyszalną dla ludzi czyli umownie w przedziale od 20Hz do 20kHz. Dźwięk będzie brzmieć dość syntetycznie, ale będzie bardziej zróżnicowany. Za pomocą tej syntezy w pewnej konfiguracji również można próbować imitować dźwięki instrumentów muzycznych, jednak mogą brzmieć dość nienaturalnie i sztucznie.

Istnieje jeszcze wiele innych rodzajów syntezy dźwięku, jednak te powyżej wymienione są istotne dla tego projektu. Możliwe jest także mieszanie tych technik ze sobą, dając bardziej rozbudowane brzmienia.

2.4.2. Budowa oscylatora

Fundamentem do działania synteźatora są oscylatory. Zasada działania takiego modułu w synteźatorze dźwiękowym jest stosunkowo prosta. Na programowy oscylator składają się informacje o: aktualnej fazie, skoku przesunięcia fazowego (częstotliwość), rodzaj generowanego kształtu sygnału okresowego, a także głośność czyli to jaką amplitudę sygnał okresowy może osiągnąć.

Podczas obliczania każdej próbki dźwiękowej synteźator musi obliczyć w jakiej fazie znajdzie się oscylator w zadanym punkcie czasu. Aby to osiągnąć dodaje do aktualnej fazy wartość skoku przesunięcia fazowego. To ile należy dodać, czyli jak duży jest skok przesunięcia fazy jest zależne od częstotliwości próbkowania, a także jaką częstotliwość oscylatora ma uzyskać. Dalej, w pewnym momencie dodanie kolejnej wartości spowoduje przepełnienie typu danych, a faza dźwiękowa zacznie być liczona od nowa. Moment przepełnienia, to moment w którym oscylator wykonał pełny swój okres czasowy.

Kolejnym etapem jest obliczenie wartości amplitudy. Aktualna faza oscylatora jest przeliczana zgodnie ze zdefiniowanym algorytmem który odpowiada za kształt fali. Następnie wartość tej amplitudy jest mnożona przez głośność, aby uzyskać wymagany maksymalny zakres amplitudy. W ten sposób powstaje próbka wynikowa oscylatora, która generuje dźwięk, albo może zostać wykorzystana przez inne komponenty synteźatora.

2.4.3. Rodzaje oscylatorów

Dla syntezy dźwięku kluczowe jest manipulacja parametrami takich oscylatorów przez inne oscylatory. Na tej podstawie można wyróżnić trzy rodzaje takich oscylatorów w zależności od sposobu pracy i zadań jakie muszą zrealizować.

HFO – to skrót od High-Frequency Oscillator, czyli oscylatory wysokiej częstotliwości. Cechują się szybkością pracy która operuje na częstotliwościach słyszalnych dla ludzi (ok. 20Hz – 20kHz). Ich zadaniem jest zazwyczaj generowanie dźwięku słyszalnego dla ludzi.

LFO – czyli Low-Frequency Oscillator, są to oscylatory które pracują dużo wolniej, bo z częstotliwością poniżej 20Hz. Te częstotliwości są już słabo słyszalne dla ludzi, ale zadaniem tych oscylatorów nie jest generowanie dźwięku, ale manipulowanie parametrami innych oscylatorów lub generatorów. To pozwala na tworzenie bardziej rozbudowanych i urozmaiconych brzmień muzycznych.

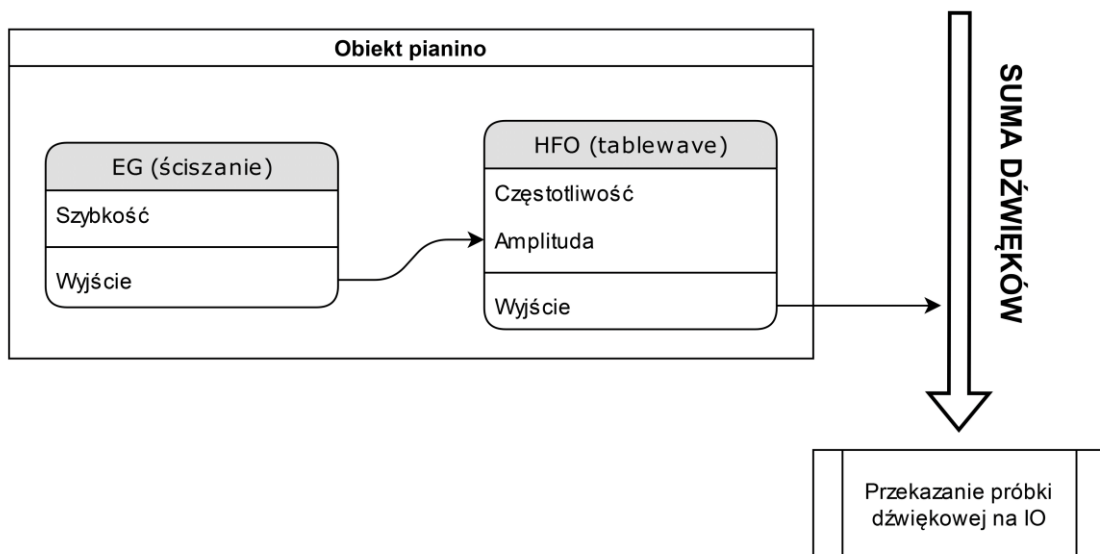
EG – Envelope Generator, inaczej generator obwiedni. Podobnie jak w przypadku LFO, ten generator ma za zadanie manipulować parametrami innych oscylatorów, głównie amplitudy. Jednak w przeciwieństwie do LFO, generator obwiedni może wykonać swój przebieg tylko raz w obrębie całego trwania dźwięku. Ma to zapewnić

imitację różnych instrumentów, które wytwarzając dźwięki tracą w ten sposób swoją energię. To z kolei powoduje że te dźwięki po prostu zanikają z czasem. EG ma więc za zadanie dostosować obwiednię dźwięku, inaczej głośność w czasie.

2.4.4. Modularność

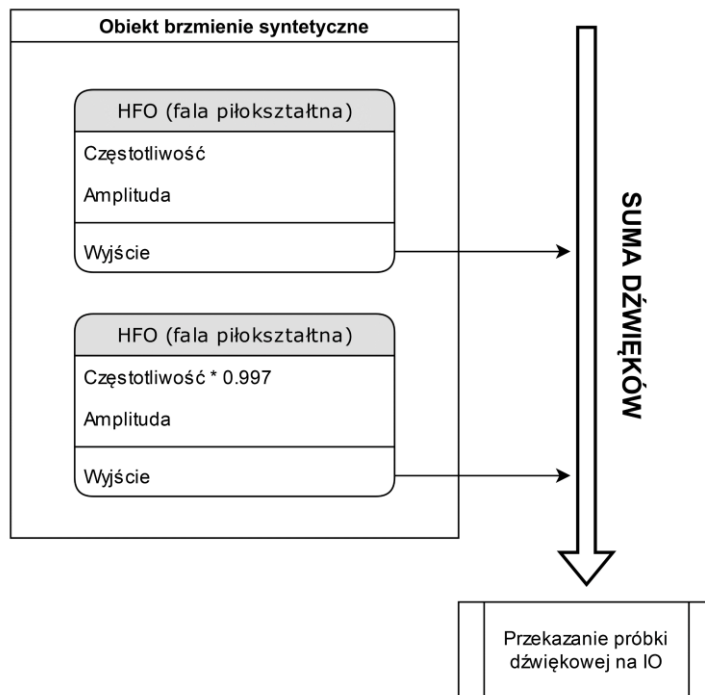
Kolejnym ważnym aspektem syntezy dźwięków jest ich modularność. Wszystkie powyżej wymienione obiekty lub moduły należy teraz ze sobą dopasować tak, aby uzyskać imitację danego instrumentu muzycznego. Należy stworzyć schemat w którym obiekty manipulują parametrami innych obiektów. Każdy schemat takiego łączenia modułów ze sobą jest typowy dla instrumentu który ma zostać odwzorowany. Potocznie taki schemat można nazwać jako preset brzmieniowy. Przykładowe ilustracje poglądowe:

- Przykład imitacji instrumentu pianina



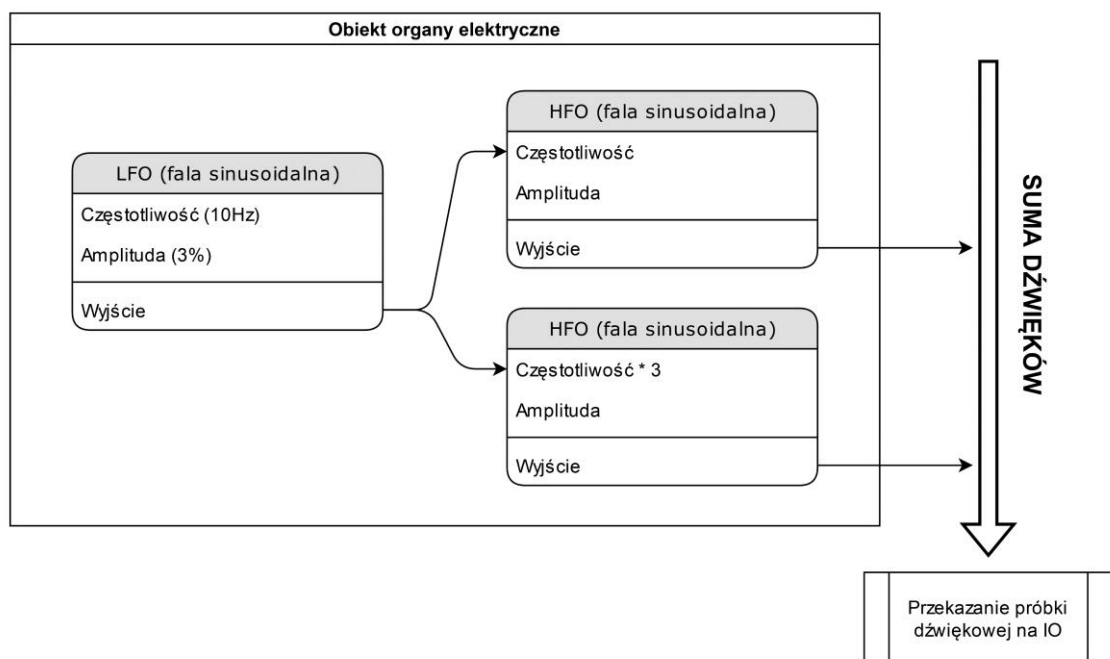
Rysunek 5: Schemat imitacji brzmienia instrumentu muzycznego (1) (źródło: grafika własna)

- Przykład dźwięku o brzmieniu syntetycznym



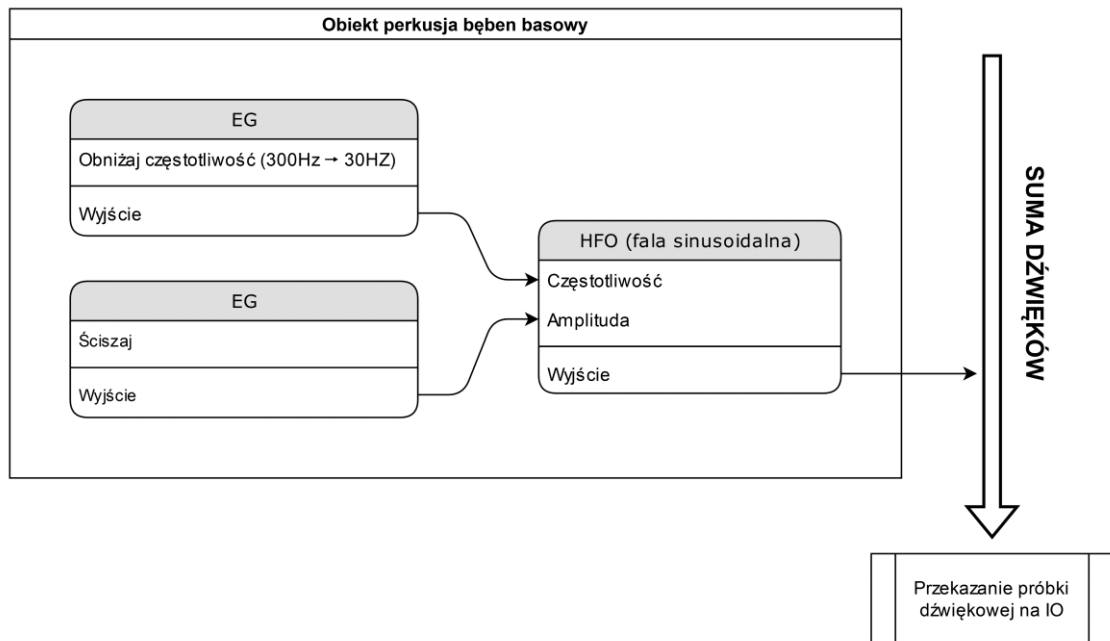
Rysunek 6: Schemat imitacji brzmienia instrumentu muzycznego (2) (źródło: grafika własna)

- Przykład instrumentu zwanego jako organy elektryczne



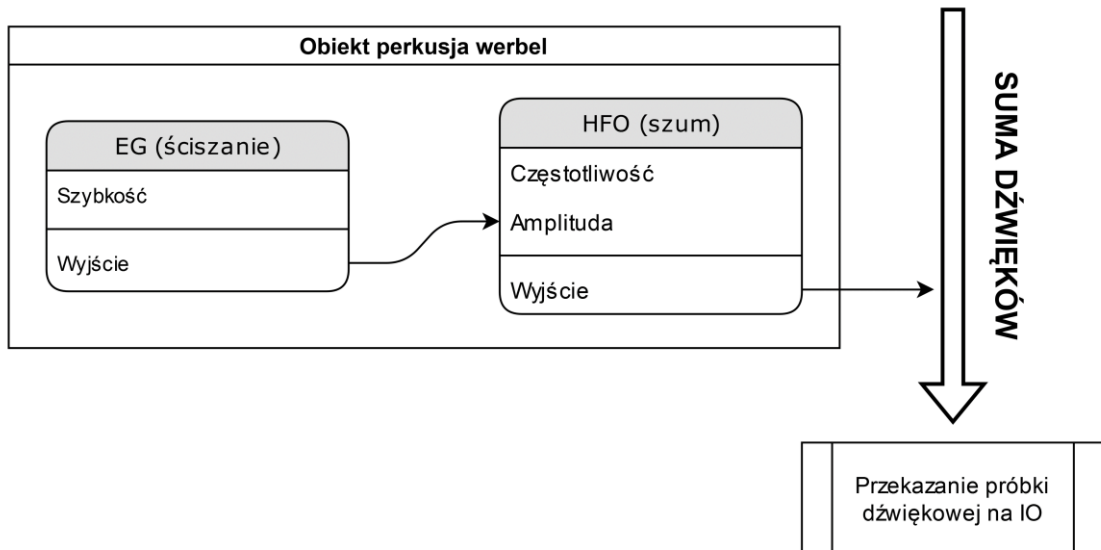
Rysunek 7: Schemat imitacji brzmienia instrumentu muzycznego (3) (źródło: grafika własna)

- Przykład imitacji instrumentu perkusyjnego – bęben basowy



Rysunek 8: Schemat imitacji brzmienia instrumentu muzycznego (4) (źródło: grafika własna)

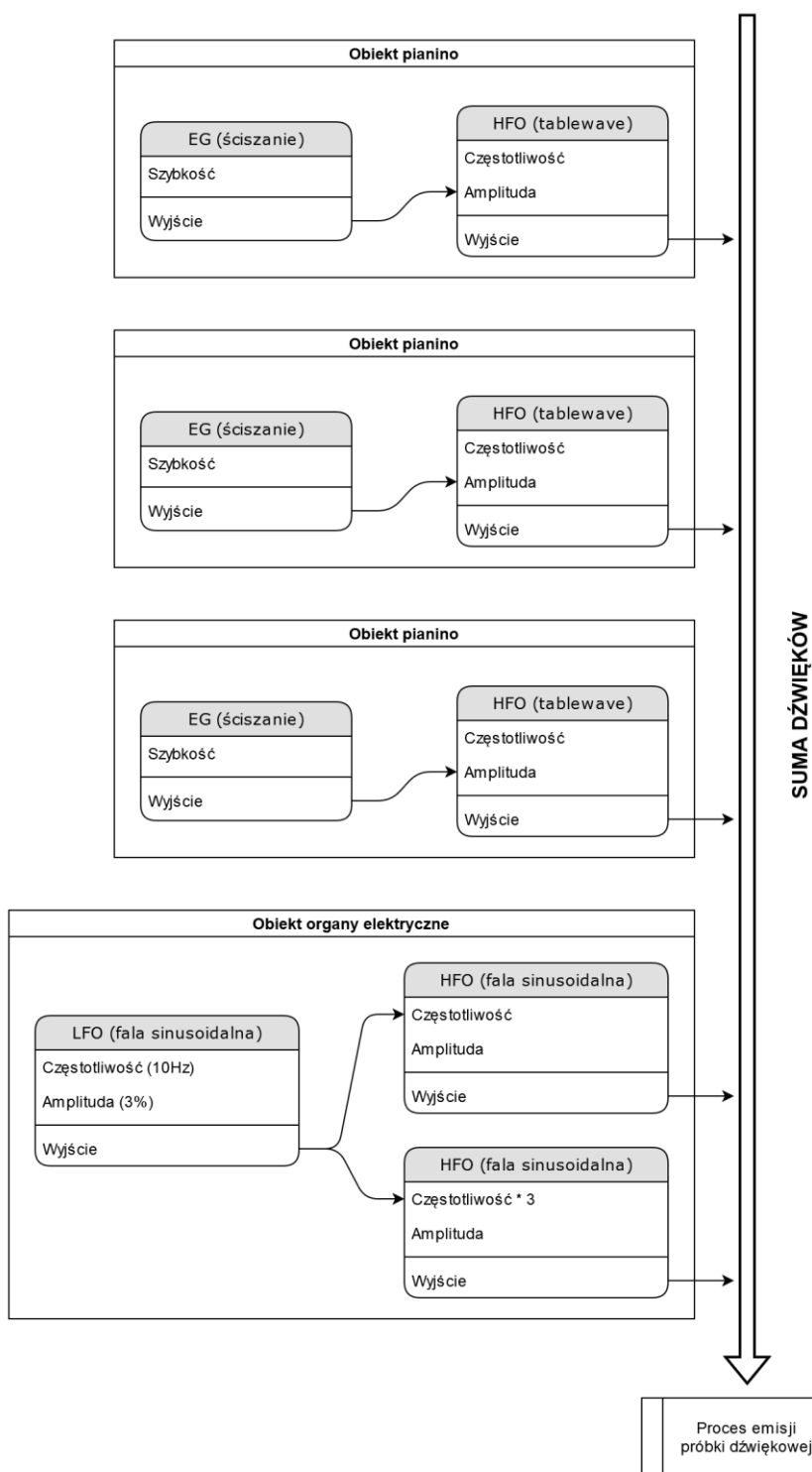
- Przykład imitacji instrumentu perkusyjnego - werbel



Rysunek 9: Schemat imitacji brzmienia instrumentu muzycznego (5) (źródło: grafika własna)

2.4.5. Opis pracy syntezy

Syntezy dźwiękowy w każdym swoim cyklu generacji próbki dźwiękowej musi zaktualizować fazy oscylatorów i obliczyć odpowiadające temu wartości chwilowe amplitud. Te wartości są następnie albo podstawą do manipulacji parametrami kolejnych generatorów, albo są przekazywane do wyemitowania poza syntezy. Aby możliwe było wygenerowanie kilku dźwięków na raz, to wartości tych próbek oscylatorów muszą zostać w pierwszej kolejności zsumowane, tak jak jest pokazane na poniższej ilustracji.



Rysunek 10: Schemat jednoczesnej pracy wielu wirtualnych instrumentów (źródło: grafika własna)

Należy jednak uważać, przekroczenie zakresu typu danych podczas sumowania spowoduje powstanie zjawiska tak zwanego przesterowaniem, co negatywnie wpłynie na jakość dźwięku. Należy więc tak dobierać zakres amplitud generowanych oscylatorów aby nie przekroczyć tej wartości.

Warto też zaznaczyć, że powyższy schemat oscylatorów nie jest stały. Podczas pracy syntezy cyfrowego generatory dźwiękowe są dynamicznie alokowane. Generatory te mogą być więc po prostu usuwane z pamięci procesora gdy nie są już potrzebne, a dodawane nowe wtedy gdy pojawi się nowy dźwięk do odtworzenia.

2.4.6. Polifoniczność syntezy

Polifoniczność to zdolność syntezy do jednoczesnego odtwarzania wielu dźwięków na raz. Niestety synteza realizowana cyfrowo ma też swoje ograniczenia dostępnych zasobów. Synteza to po prostu procesor, który ma pewną moc obliczeniową. Ta moc obliczeniowa pozwala na jednoczesną symulację wielu oscylatorów. Czym większa moc obliczeniowa, tym większa możliwa jest ilość generowanych sygnałów, co w rezultacie pozwala to na bardziej złożone kompozycje muzyczne. Ilość możliwych do jednoczesnego odtworzenia dźwięków muzycznych podawana jest w liczbie tak zwanych głosów, np. gdy synteza jest 32-głosowa oznacza to że jest zdolna do jednoczesnego odtworzenia 32 nut lub dźwięków.

3. OPIS REALIZACJI TEMATU PRACY

3.1. Studium wykonalności projektu

Studium wykonalności projektu ma przede wszystkim dać odpowiedzieć na pytanie czy projekt jest możliwy do wykonania oraz pokazać z jakimi problemami należy się zmierzyć aby taki projekt zrealizować.

3.1.1. Program testowy napisany w języku C/C++

Pierwsze podejście do tematu to próba odpowiedzenia czy mikrokontroler AVR ATmega328p z taktowaniem 16MHz jest zdolny do generacji dźwięku. Teoretycznie jest to możliwe. Zakładając, że mikrokontroler ma generować próbkę dźwiękową z częstotliwością 20kHz to okazuje się, że do obliczenia jednej próbki dźwiękowej jest do dyspozycji $16\,000\,000 / 20\,000 = 800$ cykli zegara taktującego. 800 operacji na jedną próbkę dźwiękową to nie jest dużo, ale raczej powinno być wystarczające aby wygenerować proste fale dźwiękowe.

Tak więc w celach testu zostanie napisany pierwszy, nieskomplikowany program w Arduino IDE:

```
long t = 0;

void wyemituj(int x, int y) {...}
void setup() {...}

int fsin (long time, int hz, int a)
{
    return(sin(6.28319*hz*time/20000)*a);
}

void loop()
{
    int y = fsin(t, 440, 50);
    wyemituj(t, y);
    t++;
}
```

Opis programu:

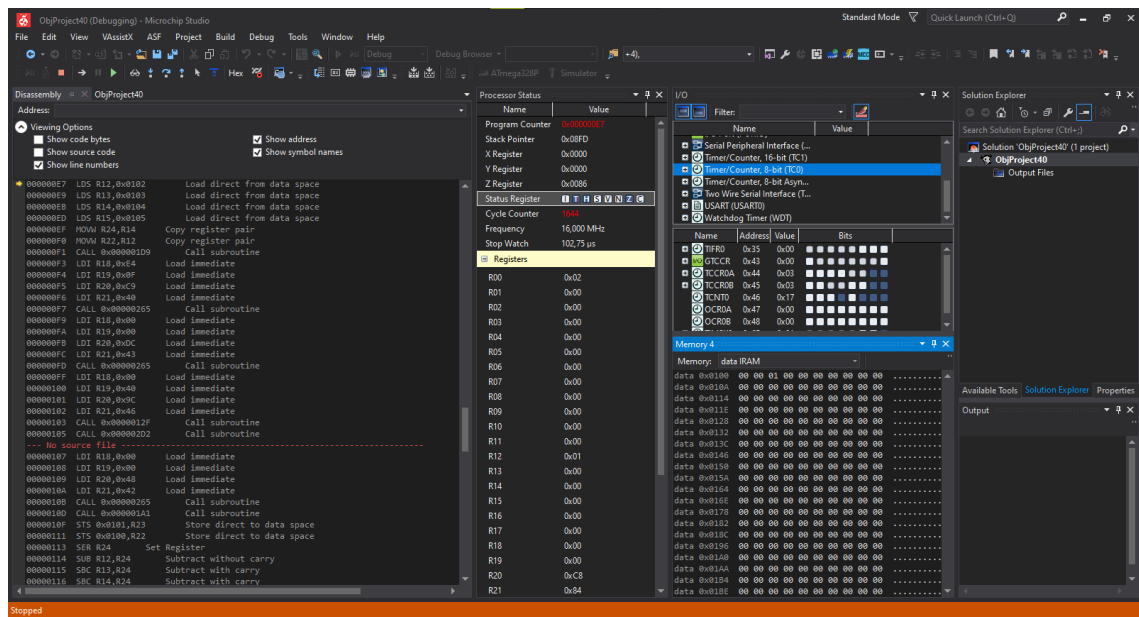
Ten program ma za zadanie obliczyć wartość chwilową w danym czasie aby odwzorować falę dźwiękową o przebiegu sinusoidalnym. Wykorzystana zostanie do tego zmienna t , która zawiera informację o numerze próbki dźwiękowej. Funkcja f_{sin} ma za zadanie z tego parametru, oraz z częstotliwości podanej w Hz obliczyć jaką powinna być to wartość, uwzględniając próbkowanie o częstotliwości 20kHz. Parametr a w tej funkcji odpowiada za maksymalną amplitudę jaką sygnał okresowy ma osiągnąć.

Następnie tak obliczona wartość z funkcji f_{sin} trafia do funkcji $wyemituj$ która zajmuje się sterowaniem pracą podłączonego głośnika. Funkcja $wyemituj$ będzie także zatrzymywać wykonywanie programu na taką ilość czasu, jaka jest wymagana do osiągnięcia częstotliwości próbkowania 20kHz. Na koniec zmienna t zawierająca numer próbki dźwiękowej zostanie zwiększona o jeden, aby w kolejnym cyklu pracy obliczyć wartość dla kolejnej próbki dźwiękowej. W ten sposób kończy się cykl pracy obliczający i generujący daną próbkę dźwiękową.

Analiza programu:

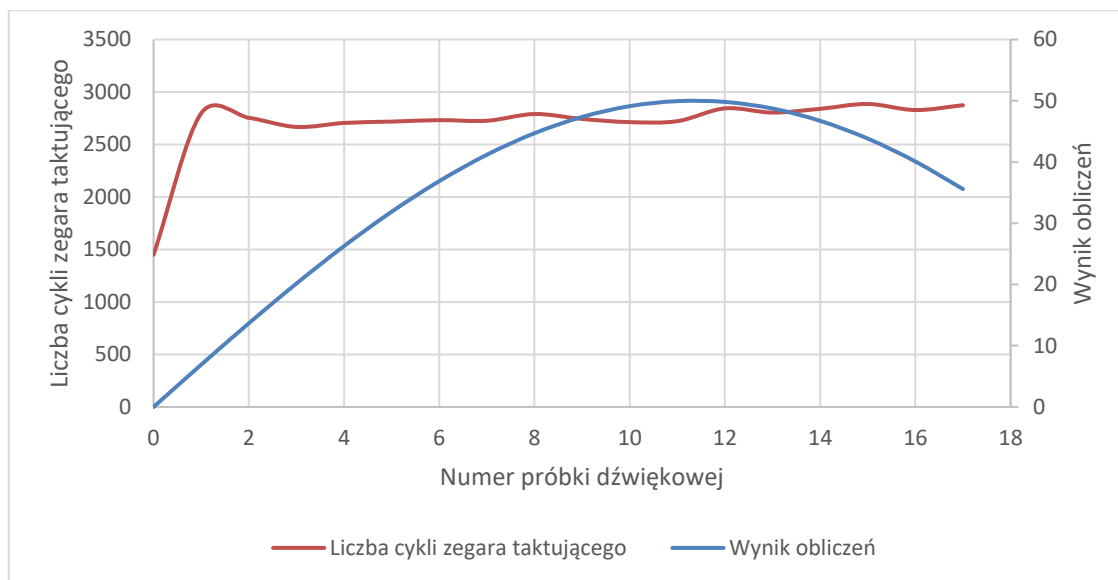
Teoretycznie taki program powinien zadziałać, w praktyce – nie. Wykonanie tych obliczeń zajmie zbyt wiele czasu! Można w symulatorze Microchip Studio dokładnie sprawdzić, jak takie obliczenia będą długo trwać:

Zrzut z ekranu symulatora:



Rysunek 11: Zrzut z ekranu programu symulującego pracę układu ATmega328P (źródło: grafika własna)

Wykres czasu trwania obliczeń:



Rysunek 12: Wykres przedstawiający czas potrzebny na wykonanie obliczeń każdej wartości funkcji sinusoidalnej (źródło: grafika własna)

Średni czas obliczenia wartości próbki dźwiękowej wynosi 2778,5 cykli zegara co odpowiada czasowi 173,65µs. Jest to zdecydowanie za dużo o ponad 3 razy! Pomijając jeszcze przy tym kwestie emisji próbki gdzie wymagane są dalsze przekształcenia próbki dźwiękowej.

Wnioski z przeprowadzonej analizy:

Wymagana jest optymalizacja kodu, jednak ciężko powiedzieć jakich zmian dokonać. Korzystając wyłącznie z języka C lub C++ trudno jest określić jak taki program będzie długo wykonywać poszczególne funkcje. Można zobaczyć w dokumentacji, że ten mikrokontroler nie posiada nawet operacji dzielenia więc wszystkie te działania są realizowane programowo. Sporo operacji mnożenia i dzielenia, a także wykonanie funkcji trygonometrycznej są bardzo czasochłonne. Dodatkowo te obliczenia będą prowadzone na liczbach zmiennoprzecinkowych które dodatkowo jeszcze bardziej zużyją zasoby mocy obliczeniowej mikrokontrolera. Brak znajomości architektury CPU przez programistę może sprawić, że kod zostanie napisany w nieefektywny sposób.

Ten powyższy program ma także jeszcze inną wadę w swoim założeniu. W utworach muzycznych bardzo często wykorzystuje się zmianę częstotliwości dźwięku w trakcie jego trwania. Gdy będzie się chciało to zrealizować w powyższej metodzie, to będzie można usłyszeć trzaski wynikające z nieciągłości fazy funkcji okresowej. Potrzebne będzie więc inne podejście, takie w którym faza funkcji oscylującej musi być pamiętana.

W ten sposób powstanie oscylator który został opisany już w części teoretycznej tej pracy i takie podejście zostanie zaimplementowane w dalszej części tego projektu.

3.1.2. Program testowy napisany w języku assembler

Drugie podejście to dalsza próba odpowiedzenia czy ten mikrokontroler jest zdolny do generacji dźwięku, jednak z nowymi założeniami. Po pierwsze językiem programowania będzie teraz assembler, który zapewni możliwość policzenia jak poszczególne części kodu długo się wykonują. Po drugie, program testowy zostanie zrealizowany na przerwaniu licznika czasu, tak aby osiągnąć precyzyjność w czasie wykonywanego kodu. Po trzecie, od teraz każda fala dźwiękowa będzie zawierała także informacje o swojej fazie. Ma to na celu przede wszystkim uprościć obliczenia, a także uniknąć zjawiska nieciągłości fazy podczas manipulacji częstotliwością fal.

Napisany program w języku AVR assembler:

```
.nolist
.include "m328pdef.inc"
.list

.dseg
czas: .byte 4

// falax - Low 16-bit pozycja fali x (faza)
// falax+1 - High 16-bit pozycja fali x (faza)
// falax+2 - Low 16-bit skok fali x (częstotliwość)
// falax+3 - High 16-bit skok fali x (częstotliwość)
// falax+4 - głośność fali x (amplituda)
fala1: .byte 5

.cseg
.org 0x0000 jmp start ; Reset
/* ... */
.org 0x0012 jmp TIM2_OVF ; Timer2 Overflow
/* ... */

start:
    //konfiguracja timerów i pinów
    /* ... */

    //konfiguracja fal
    //fala1 prostokątna
    clr r16
    sts (fala1+1), r16 //czyszczenie fali
```

```

    sts (fala1), r16    //czyszczenie fali
    ldi r19, 32
    sts (fala1+4), r19 //głośność fali
    ldi r18, 0x07
    ldi r17, 0x35
    sts (fala1+3), r18 //HB skok fali x (częstotliwość)
    sts (fala1+2), r17 //LB skok fali x (częstotliwość)

do_nothing:
    SLEEP
    rjmp do_nothing

//przerwanie zegara TIMER 2
TIM2_OVF:
    //inicjalizacja przerwania
    /* ... */

    //obliczanie fal:
    clr r2 //suma wszystkich fal

    //fala prostokątna; 19 cykli zegara
    //ładowanie danych oscylatora do rejestrów
    lds r25, (fala1+3)
    lds r24, (fala1+2)
    lds r23, (fala1+1)
    lds r22, (fala1)
    lds r1, (fala1+4)

    //obliczanie i zapisywanie pozycji fali
    add r22, r24
    adc r23, r25
    sts (fala1+1), r23
    sts (fala1), r22

    //wartości ujemne dla: pozycja fali > 50% (fala prostokątna)
    sbrc r23, 7
    neg r1

    //sumowanie fal
    add r2, r1

    /* możliwość dodania kolejnych fal w tym miejscu */

    //emisja próbki dźwiękowej
    /* ... */
reti

```

Opis programu:

Powyższy program napisany w assemblerze, choć może wydawać się długi w rzeczywistości wykonuje to samo zadanie co wcześniejszy program napisany w języku C/C++. Za pomocą dyrektywy `.dseg` została zadeklarowana zmienna *fala1* o długości 5 bajtów. Pierwsze 2 bajty jakie są przechowywane w tej zmiennej odpowiadają za zapis aktualnej fazy oscylatora. Kolejne 2 bajty odpowiadają za skok fazy fali, jest to wartość jaka jest dodawana do fazy oscylatora co każdy cykl generacji próbki dźwiękowej. Ostatni 1 bajt dotyczy wolumenu fali dźwiękowej.

Program na starcie konfiguruje mikrokontroler oraz inicjuje dane. Ustawia tryb licznika na fast PWM, włącza przerwania gdy licznik się przepełni, a także ustawia prescaler na wartość 1. Następnie inicjuje dane fali, resetując fazę i ustawiając wielkość skoku na taką która zapewni odtwarzanie częstotliwości 440Hz oraz amplitudę na 50. Dla częstotliwości 440Hz i przyjętej częstotliwości próbkowania dźwięku $16\text{MHz} / 256 / 4 = 15,625\text{kHz}$ ta wartość wyniesie $440 * 65536 / 15625 = \sim 0x0735$. Gdy nastąpi inicjalizacja, program przechodzi do trybu spoczynku i wybudza się tylko wtedy kiedy nastąpi przerwanie licznika.

Przerwanie na samym początku zajmuje się aktualizacją zmiennej *czas*. Z tej zmiennej sprawdza czy ma nastąpić synteza próbki dźwiękowej. Jeżeli zmienna *czas* jest podzielna przez 4 to nastąpi proces generacji próbki dźwiękowej, a więc wykona się jeden cykl pracy oscylatora.

Schemat cyklu pracy oscylatora w tym programie to ładowanie danych z pamięci operacyjnej do rejestrów procesora. Następnie aktualizowana jest faza oscylatora, w tym celu jest dodawana wartość skoku fazy do wartości aktualnej fazy. Ta nowa wartość fazy oscylatora jest później zapisywana w pamięci operacyjnej. W kolejnym etapie pracy oscylatora „analizowana” jest wartość fazy i na jej podstawie określana jest amplituda wynikowa oscylatora. Ta wartość jest sumowana następnie z innymi wartościami wynikowymi oscylatorów, a w tym przypadku jest dodawana do wartości 0.

Gdy już wartość próbki dźwiękowej jest już policzona to zostaje ona w dalszej części wyemitowana, np. przez sygnał PWM który działa z częstotliwością 62,5kHz a więc umożliwi nadawanie sygnału audio. W ten sposób powinien powstać dźwięk.

Analiza programu:

Analizując powyższy program można łatwo policzyć ile zajmują poszczególne funkcje oraz to jak długo kod będzie się wykonywać. Proces pracy oscylatora generującego sygnał o kształcie kwadratowym zajmuje 19 cykli zegara taktującego. Pozostałe części przerwania takie jak jego inicjacja i emitowanie sygnału, a także archiwizacja rejestrów i flag to stały koszt ok. 40 cykli zegara. Tak więc można łatwo policzyć, że w przypadku pisania kodu w assemblerze oraz z nowymi założeniami można uzyskać do 51 jednocześnie generowanych sygnałów dźwiękowych. Przeskok jest ogromny, a optymalizacja kodu „przyśpieszyła” program nawet ~180-krotnie!

Oczywiście podano tutaj dwa skrajne przypadki i ciężko porównywać obydwa kody jeden do jeden. Gdyby natomiast być bardziej fair i zwiększyć w drugim przykładzie częstotliwość próbkowania do 20kHz, oraz oscylator miałby generować sygnał o przebiegu sinusoidalnym to można realnie liczyć na 91-krotną szybkość pracy względem wcześniej napisanego kodu w C/C++. Generowanie sygnału sinusoidalnego w tym przypadku to koszt 29 cykli zegara, gdy przebieg funkcji sinusoidalnej jest zapisany w pamięci mikrokontrolera.

W ramach ciekawostki można sprawdzić jak szybki jest kod napisany w języku C względem assemblera. Gdy odwzorowano kod w języku C z zoptymalizowanego kodu w języku assembler to koszt wykonania pracy jednego oscylatora wynosił średnio 26,5 cykli zegara taktującego. Tak więc w tym przypadku język C jest ok. 1,5-krotnie wolniejszy od języka assembler. Głównie kod języka C oraz wbudowany kompilator w arduino IDE przegrywa ze względu na to, że operacje prowadzone w rejestrach są wykonywane na 2 bajtach, nawet wtedy gdy zmienne są tylko 1-bajtowe. Również możliwe jest osiągnięcie w architekturze AVR bardziej uproszczonych kombinacji operacji, które dają taki sam rezultat, a są krótsze do wykonania. Dysproporcje w szybkości działania pomiędzy tymi dwoma językami może więc jeszcze się zwiększyć. Można w tym miejscu przeprowadzić bardziej dogłębną analizę porównania szybkości języków, jednak jest to zagadnienie wykraczające poza zakres pracy inżynierskiej. Tak więc to porównanie pozostanie jedynie w ramach ciekawostki.

Należy pamiętać, że język assembler daje przede wszystkim wiedzę o architekturze mikrokontrolera, a więc w rezultacie daje to wiedzę jak skutecznie zoptymalizować kod. Tak więc biorąc ten aspekt pod uwagę można powiedzieć, że z pomocą języka assembler można liczyć na przyrost wydajności nawet 100-krotny!

Wnioski z przeprowadzonej analizy:

Podsumowując, przeprowadzona analiza jasno pokazała, że projekt jest możliwy do zrealizowania, ale warto wykorzystać przy tym język assembler. Z tego powodu pojawiają się dalsze komplikacje. Problemem jest, że język assembler oferuje bardzo niskie wsparcie bibliotek oraz materiałów dydaktycznych i dokumentacji. Również nie ma w tym języku gotowych mechanizmów znanych z języka C++ które znacząco ułatwiają pisanie kodu, a projekt będzie wymagać skalowalności, aby umożliwić proste powielanie oscylatorów. Również problemem jest brak gotowych mechanizmów dynamicznej alokacji danych. Praktycznie nie możliwym staje się też integracja tego kodu z innym, głównie przez niską popularność języka assembler. Najlepszym rozwiązaniem będzie więc połączenie języka assembler, który zapewni szybkość wykonywanej pracy, z językiem C++, który rozwiąże wszystkie wyżej wymienione problemy. Sprawdzono też, czy w języku assembler jest możliwa manipulacja i obsługa obiektów wygenerowanych przez kompilator C++ i tak, jest to jak najbardziej możliwe.

Problemem może być jedynie słaba dokumentacja lub nawet jej brak w takich zagadnieniach jak kompilator zarządza obiektami, przekazuje parametry pomiędzy funkcjami oraz jakie role przydzielono rejestrom mikrokontrolera itp. Udało się jedynie znaleźć notkę aplikacyjną o nazwie: „Atmel AT1886: Mixing Assembly and C with AVRGCC”, a to jak obiekty C++ są zarządzane przez kompilator i jak łączyć C++ z assemblerem pozostanie jedynie w kwestii inżynierii wstecznej ze względu na brak dokumentacji.

3.2. Projekt, założenia projektowe i architektura systemu

Podstawowym założeniem projektowym będzie więc możliwość symulacji pracy wielu oscylatorów jednocześnie, aby zapewnić polifoniczność muzyki. Takie oscylatory zostaną wstępnie predefiniowane, ale będzie również istnieć możliwość ich rozbudowy tak aby umożliwić zaimplementowanie swoich własnych rozwiązań. Ze zbioru tych oscylatorów powstaną kolejno obiekty dźwiękowe które będą odpowiedzialne za imitację danego brzmienia instrumentu muzycznego.

Aby jednak uruchomić te obiekty, musi istnieć środowisko które takie obiekty obsłuży. Zadaniem takiego środowiska będzie właśnie cykliczne wywoływanie zadeklarowanych metod oscylatorów aby zaktualizować ich stan. Ze względów optymalizacyjnych, mających na celu zmniejszenie użycia czasu procesora zostaną zaimplementowane dwa systemy:

- Pierwszy system zostanie oparty na przerwaniu zegara czasu mikrokontrolera. Ten system ma zapewnić możliwie jak najszybszą prędkość działania – z tego powodu przerwanie zostanie napisane w assemblerze. Również metody obiektów zawierać będą wstawki assemblerowe, a deklaracja tych obiektów będzie w C++. Mechanizmy zawarte w C++ pozwolą na łatwą, dynamiczną alokację tych obiektów w pamięci operacyjnej, a także na wielopostaciowość tych obiektów i ich skalowalność. Ten system będzie również cechować się stałą częstotliwością próbkowania która będzie liczona w kilo-hercach. Proces emisji próbki dźwiękowej zostanie także zintegrowany w tym przerwaniu.
- Drugi system będzie oparty na systemie operacyjnym czasu rzeczywistego (RTOS). Względem przerw RTOS będzie wolniejszy w działaniu, ale użycie tego systemu pozwoli mimo tego na zaoszczędzenie czasu pracy. Głównie dlatego, że oscylatory LFO i tego typu obiekty nie wymagają aż tak wysokiej częstotliwości próbkowania co w przypadku obiektów HFO. W rezultacie taki system umożliwi obiektom na dostosowanie swojej częstotliwości próbkowania w zależności od swoich potrzeb, oszczędzając tym samym wykorzystanie czasu procesora na tyle na ile jest to możliwe. Tak więc ten system cechować się będzie zmienną częstotliwością próbkowania, a także będzie też bardziej funkcjonalny. Obiekty zostaną napisane w języku C++, a system operacyjny czasu rzeczywistego zostanie zaimplementowany przez dołączenie zewnętrznej biblioteki.

Udostępniony również zostanie podstawowy interfejs programistyczny który będzie zarządzać całą pracą tego systemu. Również zostaną udostępnione pomocnicze funkcje matematyczne i stałe które będą pomocne podczas pisania kodu. Cały napisany kod będzie dołączany w postaci biblioteki tak, aby zapewnić innym użytkownikom możliwość łączenia tego kodu z innymi systemami. Będzie więc to wirtualny syntezy realizowany programowo. Z tego też powodu warto umożliwić użytkownikom konfigurację biblioteki w taki sposób, aby określić z którego timera będą generowane

przerwania oraz w jaki sposób ma być emitowany dźwięk. Jest wiele możliwości takie jak generacja sygnału PWM, sterowanie układem mostka-H, sterowanie drabinką rezystorową. Każde z tych rozwiązań będzie w następnej kolejności elektrycznie sterować membraną głośnika. W przyszłości zostaną dodane te warianty, jednak podczas pracy inżynierskiej zostanie zaimplementowane tylko rozwiązanie oparte na przerwaniu przez przepełnienie timera 2, a także sterowanie mostkiem-H który będzie w tym przypadku pełnić rolę wzmacniacza klasy D.

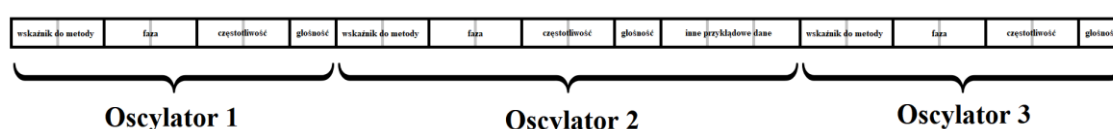
3.3.Implementacja

W tym dziale zostanie przedstawiony proces implementacji tego projektu. Na samym początku zostanie zaimplementowany system oparty na przerwaniu. Gdy ten system będzie już w pełni funkcjonalny i sprawny to w następnej kolejności zostanie rozszerzony o system oparty na RTOS.

3.3.1. Składowanie i organizacja danych

System środowiskowy wymaga do pracy zbioru oscylatorów aby mógł uruchamiać metody tych obiektów. Pytanie jest więc, jaka organizacja obiektów będzie najbardziej efektywna pod kątem szybkości wykonywanego kodu? Odpowiedź nie jest oczywista ze względu na architekturę mikrokontrolera która może sprzyjać szybkości wykonywanego kodu lub też nie. Rozważane jest rozwiązanie dla systemu działającego na przerwaniu dlatego tak kluczowa jest jego efektywna praca do działania aplikacji. Wyróżnić można dwa sposoby organizacji danych: tablica obiektów oraz lista wskaźnikowa. Każdy obiekt oscylatora może posiadać różną długość swoich danych. Warto też zaznaczyć, że system działający na przerwaniu, w każdym cyklu generujący próbkę dźwiękową musi przeiterować po każdym obiekcie i wykonać przypisaną mu metodę – z tego powodu rozważane są te sposoby implementacji organizacji danych:

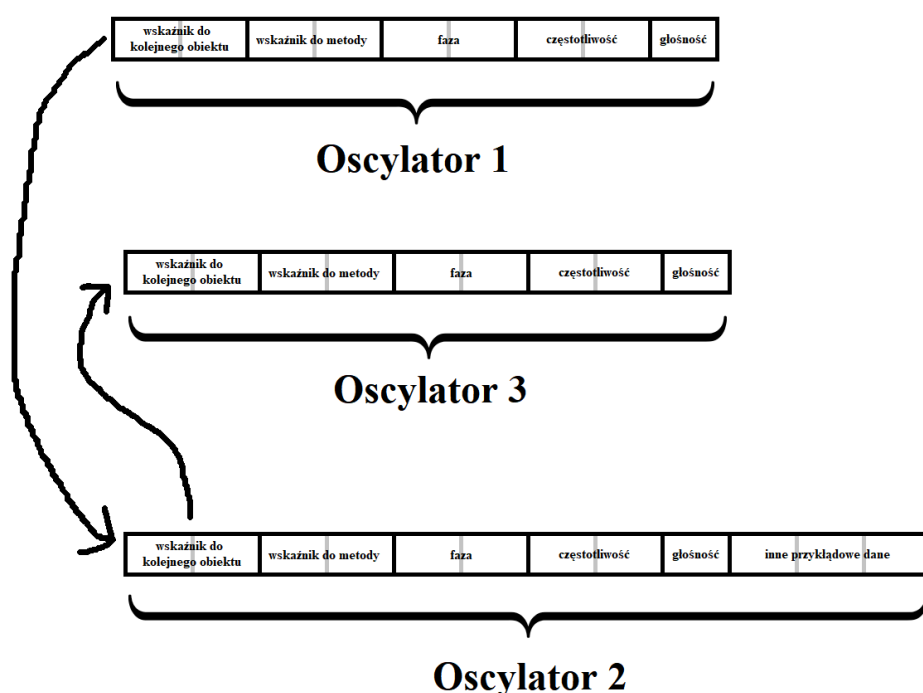
- Implementacja tablicowa – w tej metodzie organizacji danych każdy obiekt oscylatora jest umiejscowiony jeden za drugim tak jak według poniższego schematu:



Rysunek 13: Schemat przedstawiający organizację pamięci operacyjnej w mikrokontrolerze (1) (źródło: grafika własna)

Architektura mikrokontrolera za pomocą instrukcji LD ma możliwość pobierania danych z pamięci do rejestrów bajt po bajcie. Za każdym razem kiedy metoda potrzebuje załadować dane do rejestru, to wskaźnik będący w rejestrze będzie się autoinkrementować. W rezultacie, gdy metoda pobierze już wszystkie dane to automatycznie wskaźnik będzie wskazywać na kolejny obiekt – a więc zostanie zaoszczędzony czas który byłby potrzebny do załadowania tego wskaźnika z pamięci operacyjnej do rejestrów procesora.

- Implementacja listy wskaźnikowej – w tym scenariuszu każdy oscylator posiadać będzie dodatkowo wskaźnik do kolejnego obiektu.



Rysunek 14: Schemat przedstawiający organizację pamięci operacyjnej w mikrokontrolerze (2) (źródło: grafika własna)

W tym przykładzie konieczne będzie załadowanie wskaźnika z pamięci operacyjnej do rejestrów zanim będzie możliwe odczytanie danych z kolejnego obiektu. W ten sposób podczas odczytu danych w sposób iteracyjny będzie się tracić 5 cykli zegara taktującego na każdy obiekt względem poprzedniej organizacji danych.

Może się wydawać, że wybór jest prosty i że metoda tablicowa jest szybsza. Problem pojawia się jednak gdy podczas działania programu jeden z obiektów oscylatora ma zostać usunięty. Wówczas w metodzie tablicowej powstanie dziura którą będzie trzeba załatać. Najlepiej byłoby wszystkie dalsze obiekty przesunąć w lewo w miejsce dziury, ale taki proces może wymagać sporo czasu procesora w zależności od ilości danych jaką będzie trzeba przenieść. Jest to koszt co najmniej 5 cykli zegara na każdy przeniesiony 1 bajt. Dodatkowo trzeba zagwarantować ciągłość pracy, ponieważ ta struktura jest

nieustanie wykorzystywana przez przerwanie. Tak więc lepszym wyborem będzie jednak wersja z listą wskaźnikową, ale jednak ta metoda tablicowa ma potencjał do tego aby działać szybciej od metody listowej.

Metoda listowa ma z kolei jeszcze inną, dodatkową zaletę. Jest po prostu prostsza w implementacji. Wystarczy wykorzystać już istniejące mechanizmy C++ aby zaalokować lub usunąć taki obiekt z pamięci. Nie jest więc wymagane, pisanie dodatkowego kodu który będzie mieć za zadanie alokację danych w tej tablicy, również modyfikacja danych byłaby trudniejsza do osiągnięcia.

I chyba największą wadą metod tablicowych jest to, że mają już z góry ustaloną swoją wielkość. Podczas działania programu może się okazać, że potrzebna będzie większa ilość danych niż początkowo zakładano. Tak więc metoda listowa pozostanie więc ostatecznym wyborem.

Do implementacji metody listowej zostanie zadeklarowana zmienna globalna zawierająca wskaźnik do pierwszego obiektu listy jednokierunkowej. Także zostanie zaimplementowana klasa w której znajdzie się wskaźnik do kolejnego obiektu. Każda klasa oscylatora będzie dziedziczyć po tej klasie listowej. Dodatkowo ta klasa nadrzędna posiadać będzie jeszcze wskaźnik do metody obsługującej pracę obiektu, tak aby usystematyzować implementację oscylatorów. Napisany kod:

asmSynth.h

```
#include "wave.h"

extern volatile wave_null* first_wave;
```

asmSynth.cpp

```
#include "asmSynth.h"

volatile wave_null* first_wave = LAST_WAVE;
```

wave.h

```
class wave_null
{
public:
    void (*asm_func)();
    wave_null* next_wave;
};
```

3.3.2. Implementacja oscylatorów

Implementacja oscylatora jest w zasadzie stosunkowo prosta. Podstawowy oscylator zawiera informacje o swojej fazie, częstotliwości oraz głośności. Również posiada metodę która ma za zadanie aktualizować stan oscylatora oraz dodawać do sumy dźwiękowej swoje wyjście w każdym cyklu generacyjnym próbki dźwiękowej. Dodatkowo klasa oscylatora dziedziczy po klasie listowej wskaźnik do kolejnego obiektu oscylatora, oraz wskaźnik do metody obsługującej oscylator tak, aby z poziomu assemblera było możliwe wywołanie tej metody. Przykład implementacji oscylatora generującego sygnał okresowy o kształcie kwadratowym:

wave_square.h

```
#include "../wave.h"

class wave_square : public wave_null
{
public:
    int phase;
    int frequency;
    char volume;
    wave_square(int freq, unsigned char v);
    static void square_func();
};
```

wave_square.cpp

```
#include "wave_square.h"

wave_square::wave_square(int f, unsigned char v)
{
    asm_func = square_func;
    phase = 0;
    frequency = f;
    volume = v >> 1;
}

void wave_square::square_func()
{
    asm
    (
        "ldd r16, Y+4\n"
        "ldd r17, Y+5\n"
        "ldd r18, Y+6\n"
        "ldd r19, Y+7\n"
        "ldd r20, Y+8\n"

        "add r16, r18\n"
```

```

        "adc r17, r19\n"
        "std Y+4, r16\n"
        "std Y+5, r17\n"

        "sbrc r17, 7\n"
        "neg r20\n"
        "add r25, r20\n"
    );
    RETURN_WAVE;
}

```

W rejestrze Y znajduje się już uprzednio załadowany wskaźnik do tego obiektu na którym ma zostać wykonana praca. Metoda obsługująca obiekt tego oscylatora pobiera w pierwszej kolejności dane pamięci operacyjnej do rejestrów procesora. Rejestry r16 i r17 to faza oscylatora, r18, r19 to częstotliwość oraz r20 to wartość amplitudy. Następnie aktualizowana jest faza oscylatora i zapisywana z powrotem do pamięci operacyjnej do przyszłego wykorzystania. Kolejnym krokiem jest z fazy zawartej w rejestrach r16 i r17 obliczenie wartości amplitudy. Ta wartość jest następnie dodawana do sumy wszystkich dźwięków czyli do rejestru r25 (suma dla sygnałów monofonicznych). Na samym końcu za pomocą makra RETURN_WAVE następuje przejście do wykonywania metody kolejnego obiektu. To makro zostanie bardziej szczegółowo wyjaśnione w sekcji opis pracy przerwania.

3.3.3. Opis pracy przerwania

Na początku implementacji przerwania należało wybrać na którym timerze ma bazować przerwanie. Wybór padł na timer 2. Argumentami przemawiającymi za jego wybranie jest fakt, że możliwa jest jego praca w trybie asynchronicznym, tzn. możliwe jest podłączenie zewnętrznego sygnału taktującego który będzie taki timer aktywować. To może pozwolić na potencjalną synchronizację z zewnętrznymi układami, co w przypadku odtwarzania dźwięku może być przydatne. Zrezygnowano także z 16-bitowego timera 1, który co prawda umożliwiałby na sprzętowe dostosowanie częstotliwości generowanych przerwań w trybie CTC, ale taki tryb pracy timera uniemożliwia wykorzystanie go również do generowania sygnału PWM. Zrezygnowano także z timera 0 ze względu na to, że domyślnie biblioteka arduino na nim bazuje. Tak więc najlepszym wyborem do tworzenia dalszej pracy jest moim zdaniem timer 2. Zostanie on skonfigurowany w tryb fastPWM, preskaler ustawiony na wartość 1, oraz zostanie włączone przerwanie przy przepełnieniu timera.

Głównym zadaniem przerwania jest cykliczne wywoływanie metod z listy obiektów, jednak do generacji próbek dźwiękowych wymagane są także jeszcze inne zadania. Pierwszą w kolejności rzeczą jest dzielnik częstotliwości aby dostosować częstotliwość próbkowania dźwięku. Domyślnie przerwanie działa z częstotliwością 62,5kHz przy

taktowaniu mikrokontrolera 16MHz ($16\,000\,000 / 256 = 62\,500$). Taka częstotliwość może być zbyt wysoka co w rezultacie spowoduje mniejszą liczbę możliwych głosów do wygenerowania. Tak więc tę częstotliwość ogranicza się np. do liczby 20kHz. Przerwanie więc sprawdza, czy ma wykonać w tym momencie proces generacji próbki dźwiękowej, czy zakończyć przerwanie i czekać na następne wystąpienie przerwania.

core.S

```
.extern czas
.extern first_wave

//domyślnie 20khz = 62,5 * 8/25
#define licznik 8
#define mianownik 25

//.global TIMER2_OVF_vect
.global __vector_9
.global wave_end_func

//TIMER2_OVF_vect:
__vector_9:
push r24
in r24, 0x3F //SREG
push r24

//liczenie czasu
lds r24, (czas)
subi r24, licznik
brcs generuj_dzw //dzielnik częstotliwości

sts (czas), r24
pop r24
out 0x3F, r24 //SREG
pop r24
reti

generuj_dzw:
//reset czasu
subi r24, -mianownik
sts (czas), r24
sei
```

Następnym zadaniem jest już proces obliczania próbki dźwiękowej. Najpierw czyszczone są rejestry (ustawiane są na wartość 0) na których będzie dokonywać się sumy wynikowej ze wszystkich oscylatorów. Potem pobierany jest wskaźnik do pierwszego obiektu. Z tego wskaźnika wyciągany jest wskaźnik do metody obsługującej ten obiekt.

Gdy już ten wskaźnik załadowano do rejestrów to następnie wykonuje się skok do metody obsługującej dany obiekt.

core.S

```
//generowanie próbki dźwięku:
//przygotowywanie rejestrów
push r31
push r30
push r29
push r28
push r27
push r26
push r25
//push r24
push r23
push r22
push r21
push r20
push r19
push r18
push r17
push r16
push r1
push r0

clr r23 //Suma dźwiękowa sygnałów monofonicznych
clr r24 //Suma dźwiękowa sygnałów stereofonicznych - Lewy kanał
clr r25 //Suma dźwiękowa sygnałów stereofonicznych - prawy kanał

//ładowanie pierwszego wskaźnika obiektu do rejestru
lds r26, (first_wave)
lds r27, (first_wave+1)
movw r28, r26

//ładowanie wskaźnika funkcji do obsługi obiektu
ldd r30, Y+0
ldd r31, Y+1

//wykonywanie obliczeń dźwięku
ijmp
```

W tym miejscu należy się zatrzymać i dokładnie omówić dlaczego wykorzystano instrukcję skoku (ijmp) zamiast wywołanie podprogramu (icall). Jak można zauważyć w dokumentacji mikrokontrolera wykonanie skoku kosztuje o jeden cykl zegara mniej niż wykonanie skoku i zapisanie adresu powrotu na stosie.

BRANCH INSTRUCTIONS					
Mnemonics	Operands	Description	Operation	Flags	#Clocks
RJMP	k	Relative Jump	$PC \leftarrow PC + k + 1$	None	2
IJMP		Indirect Jump to (Z)	$PC \leftarrow Z$	None	2
JMP(1)	k	Direct Jump	$PC \leftarrow k$	None	3
RCALL	k	Relative Subroutine Call	$PC \leftarrow PC + k + 1$	None	3
ICALL		Indirect Call to (Z)	$PC \leftarrow Z$	None	3
CALL(1)	k	Direct Subroutine Call	$PC \leftarrow k$	None	4
RET		Subroutine Return	$PC \leftarrow STACK$	None	4
RETI		Interrupt Return	$PC \leftarrow STACK$	I	4

Rysunek 15: Fragment z dokumentacji mikrokontrolera ATmega328P (źródło: dokumentacja Atmel-42735B-ATmega328/P_Datasheet-Complete-11/2016)

Użycie instrukcji icall będzie także wymagać dodatkowo instrukcji ret która powróci do miejsca wywołania funkcji icall. Jest to nie potrzebne. Zamiast powracać z metody do miejsca w którym tą metodę wywołano, żeby ponownie przejść do kolejnego obiektu, to lepszym rozwiązaniem jest bezpośrednio przejście do kolejnej metody obiektu wykonującej obliczenia. Będzie to kosztować o 5 cykli zegara mniej. Tak więc każda metoda obsługująca obiekt zamiast instrukcji ret musi zawierać zestaw instrukcji która umożliwi przejście do kolejnego obiektu. Zadeklarowano więc makro RETURN_WAVE które ma zastąpić standardowy ret jako powrót z funkcji. To makro ma za zadanie ułatwić dodawanie na końcu metody fragmentu kodu, który będzie odpowiedzialny za przejście do kolejnej metody kolejnego obiektu:

wave.h

```
//ładowanie wskaźnika do kolejnego obiektu i wykonanie programu obiektu
//X = Y->next_obj;
//Y = X;
//Z = Y->func();
//go to Z;
#define RETURN_WAVE \
asm \
( \
    "ldd r26, Y+2\n" \
    "ldd r27, Y+3\n" \
    "movw r28, r26\n" \
    "ldd r30, Y+0\n" \
    "ldd r31, Y+1\n" \
    "ijmp\n" \
)
```

Te metody będą przechodziły do kolejnych obiektów w zasadzie bez limitu. Należy więc teraz dodać mechanizm który zakończy pracę gdy dotrze się już do ostatniego elementu. Nie będzie informacji o rozmiarze listy. Lepszym rozwiązaniem będzie zastosowanie specjalnego znaku końca listy. Jednak zamiast nakładać specjalny symbol

na wskaźnik ostatniego elementu, który musiałby być też sprawdzany po każdym wykonaniu metody obiektu, lepszym rozwiązaniem jest stworzyć specjalny obiekt z wskaźnikiem do dalszej części kodu przerwania. W ten sposób automatycznie zakończy się iteracja po obiektach oscylatorów i nastąpi powrót do wykonywania dalszej części przerwania.

wave.h

```
class wave_end : public wave_null
{
public:
    wave_end();
};

extern "C" {void wave_end_func();}

static wave_end obj_wave_end;
#define LAST_WAVE (&obj_wave_end)
```

wave.cpp

```
#include "wave.h"

wave_end::wave_end()
{
    asm_func = wave_end_func;
}
```

asmSynth.cpp

```
volatile wave_null* first_wave = LAST_WAVE;
```

core.S

```
.global wave_end_func

wave_end_func:
    //dalsza część przerwania
```

Ostatnim krokiem pracy przerwania to emisja próbki dźwiękowej. W zależności od przyjętego schematu połączenia mikrokontrolera z głośnikami ta sekcja kodu może się różnić. Mikrokontroler może generować sygnał PWM, sterować mostkiem-H który pracował by bezpośrednio jako wzmacniacz do głośników, lub być podłączony do drabinki rezystorowej w celu jak najwierniejszego odwzorowania dźwięku. Możliwości jest wiele więc z tego powodu będą zachodzić różne przekształcenia próbki dźwiękowej aby ją wyemitować.

Dla przykładu pokazano poniżej kod sterujący mostkiem-H użyty jako wzmacniacz do głośników. Rozwiązanie to działa na podobnej zasadzie co wzmacniacz klasy D.

Mostek-H jest sterowany za pomocą sygnału PWM. Sygnał PWM z kolei zadziała w tym przypadku jako cyfrowy przetwornik DAC.

core.S

```
wave_end_func:
add r23, r25 //sygnał mono miksowany do Lewego kanału stereo
add r24, r25 //sygnał mono miksowany do prawego kanału stereo
clr r0
cli

//proces emisji sygnału stereofonicznego
//wystawienie wyniku obliczeń na io do wyemitowania przez PWM
//adresy:
//0x88 - OCR1AL
//0x8A - OCR1BL
//0xB3 - OCR2A
//0xB4 - OCR2B

//Lewy kanał
sbrs r24, 7
rjmp 1f

com r24 //przebieg ujemny
lsl r24
sts 0x8A, r24
sts 0x88, r0
rjmp 2f

1:
lsl r24 //przebieg dodatni
sts 0x88, r24
sts 0x8A, r0

2:
//prawy kanał
sbrs r23, 7
rjmp 1f

com r23 //przebieg ujemny
lsl r23
sts 0xB4, r23
sts 0xB3, r0
rjmp 2f

1: //przebieg dodatni
lsl r23
sts 0xB3, r23
sts 0xB4, r0
```

```

2:
//koniec przerwania, przywracanie rejestrów
pop r0
pop r1
pop r16
pop r17
pop r18
pop r19
pop r20
pop r21
pop r22
pop r23
//pop r24
pop r25
pop r26
pop r27
pop r28
pop r29
pop r30
pop r31

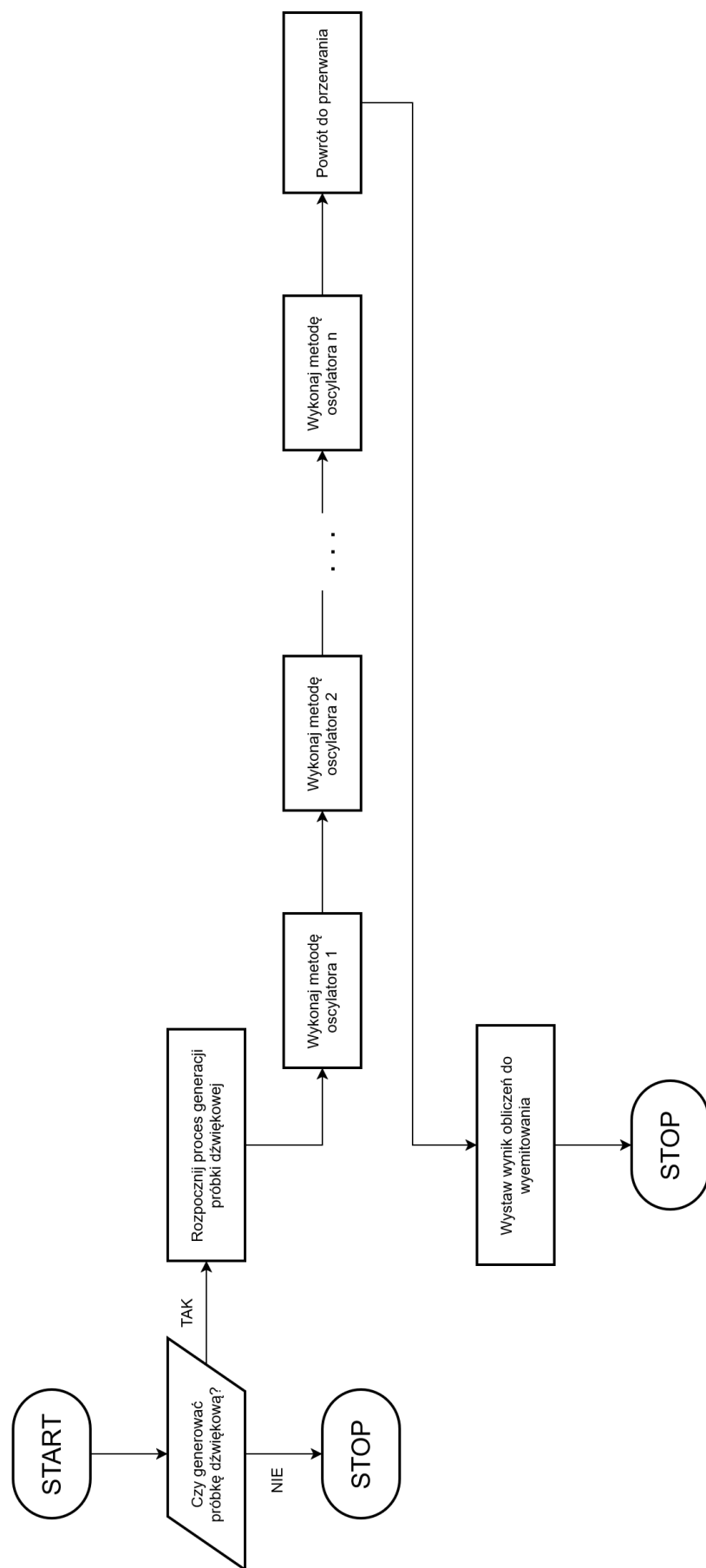
pop r24
out 0x3F, r24 //SREG
pop r24
reti

```

Przydział wykorzystywanych rejestrów:

Z – Adres funkcji
 Y – Adres obiektu (stały, nie można zmieniać)
 X – Adres obiektu (kopia, można modyfikować)
 R25 – Suma dźwiękowa sygnałów monofonicznych
 R24 – Suma dźwiękowa sygnałów stereofonicznych – lewy kanał
 R23 – Suma dźwiękowa sygnałów stereofonicznych – prawy kanał
 R16-R22 – Do dyspozycji obliczeń oscylatorów
 R1 – wynik z mnożenia +1
 R0 – wynik z mnożenia

W celu łatwiejszego zrozumienia kodu poniżej znajduje się schemat blokowy tego przerwania:



Rysunek 16: Schemat blokowy przerwania (źródło: grafika własna)

3.3.4. Interfejs programistyczny

Na obecny stan systemu interfejs programistyczny ogranicza się do operacji dodawania i usuwania elementów z listy.

asmSynth.h

```
#include "wave.h"

extern volatile char czas;
extern volatile wave_null* first_wave;

void addWave(wave_null*);
void pushBackWave(wave_null*);
wave_null* getPrevious(wave_null*);
void deleteWave(wave_null*);
void deleteAllWave(void);
```

asmSynth.cpp

```
#include "asmSynth.h"

volatile char czas = 0;
volatile wave_null* first_wave = LAST_WAVE;

void addWave(wave_null* w)
{
    w->next_wave = (wave_null*)first_wave;
    first_wave = w;
}

void pushBackWave(wave_null* w)
{
    w->next_wave = LAST_WAVE;
    getPrevious(LAST_WAVE)->next_wave = w;
}

wave_null* getPrevious(wave_null* w)
{
    wave_null* p = (wave_null*)first_wave;
    while(p->next_wave != w && p->next_wave != LAST_WAVE)
        p = p->next_wave;
    return p;
}

void deleteWave(wave_null* w)
{
    if (w == first_wave)
        first_wave = w->next_wave;
```

```

else
    getPrevious(w)->next_wave = w->next_wave;
delete w;
}

void deleteAllWave()
{
    wave_null* w = (wave_null*)first_wave;
    first_wave = LAST_WAVE;
    while(w != LAST_WAVE)
    {
        wave_null* n = w->next_wave;
        delete w;
        w = n;
    }
}

```

3.3.5. Funkcje pomocnicze

Zestaw funkcji i makr będą mieć za zadanie pomóc przy przeliczaniu różnych typów jednostek, a także zdefiniować różne stałe.

Ta definicja ma za zadanie obliczenie panującej w projekcie częstotliwości próbkowania dźwięku, podawana jest w jednostkach Hz. F_CPU to stała nadawana przez kompilator i oznacza częstotliwość taktowania mikrokontrolera. Licznik i mianownik to wartości jakie zostały użyte przy przerwaniu aby zredukować jego częstotliwość.

```
#define SAMPLE_RATE (F_CPU/256*licznik/mianownik)
```

Ta funkcja ma za zadanie z podanej wartości częstotliwości Hz przeliczyć do wartości skoku jaką oscylator ma dodawać w każdym cyklu swojej pracy.

```

int hz(double hz)
{
    return hz*65536/SAMPLE_RATE;
}

```

Ta funkcja ma za zadanie z numeru klawisza instrumentu muzycznego w standardzie midi obliczyć częstotliwość jaka odpowiada temu dźwiękowi.

```

#include <math.h>

#define TUNE 440 //częstotliwość strojenia
#define TONE_INTERVAL 2 //wielkość oktawy
#define NUMBER_OF_TONES 12 //ilość dźwięków na oktawę
#define KEY_SHIFT 69 //numer klawisza, w standardzie midi dźwięk a4 = 69

```

```
double keyboard(double key)
{
    key = TUNE*pow(TONE_INTERVAL, ((key-KEY_SHIFT)/NUMBER_OF_TONES));
    return key;
}
```

Powyższą funkcję można dodatkowo rozszerzyć o parametr nazywany centem.

```
double keyboard(double key, double cent)
{
    key = keyboard(key+cent/100);
    return key;
}
```

Również jest możliwość wpisywania częstotliwości dźwięków bezpośrednio jako nazwy klawiszy instrumentu. Poniżej przedstawiono fragment definicji tylko dla jednej oktawy.

```
#define c4 261.6255653006
#define cs4 277.1826309769
#define d4 293.6647679174
#define ds4 311.1269837221
#define e4 329.6275569129
#define f4 349.228231433
#define fs4 369.9944227116
#define g4 391.9954359817
#define gs4 415.3046975799
#define a4 440
#define as4 466.1637615181
#define b4 493.8833012561
#define h4 493.8833012561
```

3.3.6. Rozszerzenie projektu o system RTOS

Kiedy pierwszy system jest już w pełni funkcjonalny, przetestowany i sprawny można przejść do implementacji systemu RTOS. Na tym systemie ma zostać oparta budowa całej reszty oscylatorów i innych tego typu obiektów. System będzie uruchamiać zadania oscylatorów aby zaktualizować ich stan, ale w przeciwieństwie do systemu opartym na przerwaniu będzie to robić znacznie rzadziej co pozwoli zaoszczędzić użycie czasu procesora. Każde zadanie będzie mogło sobie dowolnie wybrać po jakim czasie ma nastąpić ponowna kontynuacja swojego zadania. Takie założenie pozwoli zmaksymalizować wydajność pisanego kodu.

W celu implementacji systemu RTOS zostanie wdrożone gotowe rozwiązanie w postaci biblioteki FreeRTOS. Z przeprowadzonych przeze mnie badań wynika, że obecnie jest to jedyny system operacyjny czasu rzeczywistego, który został przeportowany na architekturę AVR, a konkretnie na mikrokontroler ATmega328P. Być

może pojawią się wkrótce gotowe inne rozwiązania jak np. MicroC/OS-II, mogące dać lepsze możliwości implementacji kodu. Bibliotekę FreeRTOS można znaleźć pod linkiem: https://github.com/feilipu/Arduino_FreeRTOS_Library. Wersja systemu użyta w tej pracy to 11.1.0-2 wydana na licencji MIT.

3.3.6.1. Implementacja biblioteki FreeRTOS

Na samym wstępie zostanie zaimplementowana klasa mająca za zadanie imitację dźwięku instrumentu pianina. Zadaniem tej klasy jest stworzenie nowej instancji oscylatora HFO o kształcie kwadratowym i stopniowe jego ściszenie. Po zakończeniu swojego działania, tj. wyciszenie dźwięku do wartości bliskiej zeru, instancja klasy pianino ma za zadanie usunąć się aby zwolnić zasoby mikrokontrolera. Implementacja tej klasy wygląda następująco:

```
#include <Arduino_FreeRTOS.h>

class piano
{
public:
    TaskHandle_t TaskHandle;
    int frequency;
    int volume;

    static void play_piano(void* pvParameters)
    {
        piano* p = (piano*)pvParameters;
        auto* w = new wave_square(hz(p->frequency), p->volume);
        addWave(w);
        TickType_t xLastWakeTime = xTaskGetTickCount();
        const TickType_t xFrequency = 1;

        for(int i = p->volume; i > 10; i = i*0.95)
        {
            w->volume = i/10;
            vTaskDelayUntil(&xLastWakeTime, xFrequency);
        }

        deleteWave(w);
        TaskHandle_t th = p->TaskHandle;
        delete p;
        vTaskDelete(th);
    }

    piano(int f)
    {
        frequency = f;
        volume = 50 * 10;
    }
};
```

```

    xTaskCreate(piano::play_piano, // Task function
               "p", // Task name
               70, // Stack size
               (void*)this, // Function parameters
               1, // Priority
               &TaskHandle); // Handle
}
};

```

Poniższy program ma za zadanie przetestować działanie klasy `piano`. W tym celu zostanie odegrana prosta melodia. Czas trwania dźwięku jest na tyle długi, że te dźwięki zaczynają się nakładać jeden na drugi. Napisany program testowy w Arduino IDE:

```

#include <Arduino_FreeRTOS.h>
#include <asmSynthAllFiles.h>

void zagraj_melodie(void* v)
{
    TickType_t xLastWakeTime = xTaskGetTickCount();
    int const t = 25;
    bool w = false;
    while(1)
    {
        new piano(g4);
        vTaskDelayUntil(&xLastWakeTime, t);
        new piano(e4);
        vTaskDelayUntil(&xLastWakeTime, t);
        new piano(e4);
        vTaskDelayUntil(&xLastWakeTime, t);

        new piano(f4);
        vTaskDelayUntil(&xLastWakeTime, t);
        new piano(d4);
        vTaskDelayUntil(&xLastWakeTime, t);
        new piano(d4);
        vTaskDelayUntil(&xLastWakeTime, t);

        new piano(c4);
        vTaskDelayUntil(&xLastWakeTime, t/2);
        new piano(e4);
        vTaskDelayUntil(&xLastWakeTime, t/2);
        if(w == false)
        {
            w = true;
            new piano(g4);
        }
        else
        {

```



```

        w = false;
        new piano(c4);
    }
    vTaskDelayUntil(&xLastWakeTime, t*2);
}
}

void setup()
{
    asmSynthSetup();

    xTaskCreate(zagraj_melodie, // Task function
               "m", // Task name
               60, // Stack size
               NULL,
               1, // Priority
               NULL);
}

void loop() {}

```

3.3.6.2. Analiza rezultatów implementacji FreeRTOS

Teoretycznie ten program działa i rzeczywiście możliwe jest odtwarzanie wiele dźwięków jednocześnie. Praktycznie jednak liczba możliwych oscylatorów LFO do jednoczesnego odtworzenia jest niska bo wynosi około 13. Jest to wartość zdecydowanie niższa od oczekiwanej. System oparty na przerwaniu, symulujący oscylatory HFO bez problemu może odtworzyć już większą ilość bo nawet 16 sztuk, a ilość planowanych oscylatorów LFO do jednoczesnej symulacji przez system operacyjny czasu rzeczywistego to nawet około 50. Tak więc ten aspekt, czyli poprawa wydajności systemu operacyjnego czasu rzeczywistego zostanie dalej rozwinięty w przyszłości.

Głównym problemem FreeRTOS jest ilość alokowanej pamięci operacyjnej. Gdy rozmiar stosu zostanie zbyt bardzo zmniejszony to następować będzie wyciek pamięci co w rezultacie spowoduje niestabilność systemu. Gdy rozmiar stosu z kolei będzie zbyt wysoki, to ilość możliwych zadań do jednoczesnego odtworzenia spadnie. Dodatkowo zadania są dynamicznie alokowane, a nie na stałe co jeszcze bardziej zwiększa ilość realnie zajętej pamięci. Powyższy kod zawiera taką konfigurację która najlepiej dostosowuje wielkość stosu przy zachowaniu stabilności systemu. Żeby cel był możliwy do osiągnięcia należy wykonać kolejną optymalizację użycia zasobów tego mikrokontrolera, czyli obniżenie wykorzystywania pamięci operacyjnej.

Niestety architektura tego procesora nie za bardzo wspiera przełączanie kontekstu. 32 rejestry które należy zapisać aby możliwe w ogóle było przełączenie kontekstu to koszt który musi ponieść pamięć operacyjna za każde uruchomione zadanie. Dodatkowo w implementacji FreeRTOS należy tej pamięci udzielić jeszcze więcej na alokację stosu,

tak aby przerwania które wystąpią i które przerwą jedno z wielu zadań mogły odłożyć zawartość rejestrów na każdy z tych stosów. Moim zdaniem jest to niepoprawna implementacja stosu w tej bibliotece, dająca także łatwą możliwość na potencjalne wycieki pamięci, a w rezultacie awarię systemu.

Wniosek jest jeden – aby było możliwe wygenerowanie tak dużej liczby oscylatorów należy pozbyć się pamiętania kontekstu wykonywanego zadania! A przynajmniej ograniczyć w znacznym stopniu.

Biblioteka FreeRTOS ma także jeszcze inne problemy w swojej architekturze. Również ta biblioteka zajmuje sporo miejsca pamięci programowej bo około 10kb. Sama deklaracja nowych zadań wymaga sporo niepotrzebnych parametrów jak np. nazwa zadania. Ten system został zaprojektowany z myślą o zupełnie innym zastosowaniu.

Podsumowując architektura tego systemu jest słabo zoptymalizowana pod ten mikrokontroler. Wobec tego ten system warto zastąpić przez własne, dedykowane rozwiązanie które w założeniach będzie bardziej sprzyjające dla architektury AVR. Poniżej podano założenia jakie system operacyjny czasu rzeczywistego powinien mieć, aby dobrze działać w zastosowaniu pracy syntezy dźwiękowego na tym mikrokontrolerze.

3.3.6.3. Założenia projektowe dedykowanego systemu RTOS

Tak więc na podstawie przedstawionych wcześniej problemów zostanie zaprojektowane własne rozwiązanie które w przyszłości zostanie wdrożone zastępując implementację FreeRTOS. Taki projekt dostarczy założenia jakie system operacyjny czasu rzeczywistego powinien spełniać aby poprawić działanie całej aplikacji.

Głównym celem jest zmniejszenie ilości wykorzystywanej pamięci operacyjnej, a także szybkość działania. Jednym z przedsięwzięć będzie więc usunięcie kontekstu zadania, jednak każde z zadań potrzebuje danych na których ma pracować. Wobec tego lepszym rozwiązaniem będzie utworzenie obiektów do przechowywania tych danych, a także metody które będą na tych obiektach pracować. To rozwiązanie także pomoże w skalowalności tych obiektów które jest szczególnie potrzebne w tym projekcie.

Tak więc zostanie stworzona klasa abstrakcyjna po której będą dziedziczyć klasy z zadaniami.

```
class run
{
public:
    virtual unsigned int running();

    run* next_obj;
    long time_to_run;

    run();
    virtual ~run();
}
```

```
void start_running();  
};
```

Wszystkie zadania będą ułożone w strukturze kolejki priorytetowej posortowanej od zadania które ma najwcześniej się uruchomić do zadania które zostanie najpóźniej uruchomione. Nie jest wymagane ustalanie priorytetów zadania, których sprawdzanie, a następnie zarządzanie wymagać będzie kolejnych zasobów mocy obliczeniowej mikrokontrolera. Lepiej jest ten czas przeznaczyć na wykonywanie tych zadań. Filozofia tego systemu jest taka, że każde zadanie będzie wykonywać się na tyle szybko, że nie będzie opłacalne przełączanie się kontekstu ponieważ może to często potrwać dłużej niż wykonanie samego zadania.

Za pomocą wirtualnej metody running możliwe będzie zadeklarowanie zadania przez klasę podrzędną. Tą metodę będzie uruchamiać scheduler w zleconym czasie. Metoda ta w trakcie swojej pracy może zwrócić wartość która ma oznaczać dla schedulera czy zadanie ma zostać usunięte, czy wywołane ponownie po określonym czasie. Metoda start_running jest metodą, która zapewnia dodanie do kolejki nowego zadania. Tak więc można będzie skonfigurować tak obiekt, że samo stworzenie instancji spowoduje już pracę zadania, które następnie samo się zakończy gdy określone zadanie zostanie zrealizowane.

Można już teraz zauważyć, że możliwe jest stworzenie systemu wykonującego zadania bez użycia kontekstu. Jednak takie podejście może nie wystarczyć i utrudnić pisanie kodu zwłaszcza dla zadań które wymagają sekwencyjnego działania, a nie powtarzalnych czynności. Można więc spróbować powyższą klasę dodatkowo rozszerzyć o wskaźnik, na którym będzie dynamicznie alokowany kontekst wykonywanego zadania. Kontekst zadania, czyli zawartość rejestrów, stosu i wskaźnik programu PC, będzie alokowana dynamicznie tylko wtedy, kiedy będzie to rzeczywiście potrzebne, czyli np. wtedy gdy zadanie wywoła funkcję opóźniającą po której ma nastąpić dalsza, inna część kodu. Sama dynamiczna alokacja pozwoli także zaoszczędzić pamięć ponieważ będzie zarezerwowane dokładnie tyle pamięci ile rzeczywiście zadanie jej wykorzystało. Algorytm zachowania kontekstu wyglądałby następująco. Po przerwaniu toku pracy zadania zostanie zapisany wskaźnik programowy PC, wszystkie rejestry, a także stos programu. Co najważniejsze, ten stos zostanie zdjęty z głównego stosu i zostanie zapisane dokładnie tyle tych wartości, ile zadanie odłożyło na nim wartości. Jest to rozwiązanie znacznie oszczędniejsze w pamięć, a przy tym nieznacznie wolniejsze od implementacji FreeRTOS w której należy odpowiednio tej pamięci więcej przydzielić tak aby z tego stosu mogły korzystać także przerwania. Moim zdaniem FreeRTOS ma niepoprawnie zaimplementowaną alokację w pamięci kontekstu zadania, która jest zasobożerna, a także sprzyja wyciekom pamięci zwłaszcza dla systemów wykorzystujących przerwania.

Pozostałe cechy takie jak synchronizacja czy przekazywanie parametrów między zadaniami można próbować odwzorować z rozwiązań innych systemów operacyjnych.

4. ANALIZA WYNIKÓW

Podsumowując, projekt udało się zrealizować. Mikrokontroler AVR ATmega328P, chociaż nie jest dedykowany do tego typu zastosowań ponieważ nawet nie posiada wbudowanego konwertera DAC, to możliwe jest na nim syntezywanie i odtwarzanie muzyki polifonicznej. Udało się na nim zbudować środowisko programistyczne które jest w stanie symulować pracę wielu oscylatorów jednocześnie, a następnie w ten sposób wyliczać próbki dźwiękowe i je emitować. Zapewniono także wsparcie do implementacji własnych niestandardowych typu obiektów oscylatorowych przez zastosowanie mechanizmów dziedziczenia znanego z programowania obiektowego. Z powodzeniem także dodano do projektu bibliotekę FreeRTOS jednak okazało się, że ten system nie będzie idealnym rozwiązaniem dla tego projektu.

Do poprawienia jakości działania warto więc zaimplementować swoje własne, dedykowane rozwiązanie systemu operacyjnego czasu rzeczywistego według podanych wcześniej założeń. Kolejną dość istotną rzeczą którą warto jeszcze zaimplementować jest dodanie mechanizmu zapobiegającego przeciążeniu mocy obliczeniowej procesora. Aktualnie do systemu z wykorzystaniem przerwania można zlecić do wygenerowania na tyle dużą liczbę oscylatorów, że mikrokontroler nigdy tego przerwania nie zdąży zakończyć przed wystąpieniem kolejnego cyklu generacji próbki dźwiękowej. W rezultacie mikrokontroler po prostu zawiesi się. Warto pomyśleć o zabezpieczeniu na wypadek takiego scenariusza. Kolejną opcją do rozważenia jest zastosowanie bufora próbek dźwiękowych w celu zwiększenia szybkości pracy mikrokontrolera. Pozwoli to zaoszczędzić czas przeznaczony na alokowanie rejestrów na stosie podczas przerwania. Zamiast archiwizować rejestry co każdy proces generacji próbki dźwiękowej można to zrobić raz, ale na kilka próbek dźwiękowych. Warto też pomyśleć o technice ditheringu która mogłaby polepszyć jakość odtwarzanego dźwięku.

5. PODSUMOWANIE

Podsumowując, realizacja tego projektu okazała się być bardzo trudna. Proces polifonicznej syntezy dźwięku to proces bardzo złożony, zwłaszcza jak dla takiego prostego mikrokontrolera jakim jest AVR ATmega328P. Należało się wykazać dużą znajomością w programowaniu, a także posługiwać się licznymi dokumentacjami. Zastosowane techniki w postaci łączenia kodu assemblerowego z kodem C++ zaowocowała pozytywnymi rezultatami w działaniu aplikacji. Możliwość dołączania kodu jako biblioteki sprawia, że projekt może być implementowany wraz z innymi systemami na jednym chipie.

Ta praca również pokazuje jak istotne znaczenie dla jakości działania kodu ma język assembler. Nie dlatego żeby w tym języku pisać programy, ale dlatego aby wiedzieć jak nie pisać programów w językach wyższego poziomu. Znajomość assemblera może być kluczowa dla zrozumienia zasady działania mikrokontrolerów co pomoże w pisaniu wydajnego kodu.

Projekt został udostępniony na platformie github dla szerszej publiczności – kod można znaleźć pod linkiem: <https://github.com/sokol00/SyntezatorArduino>

6. BIBLIOGRAFIA

- [1] „Forum internetowe diyaudio.pl,” 11 styczeń 2011. [Online]. Available: <https://diyaudio.pl/articles/archiwum-audiomanii/639029-dzwiek-co-jak-gdzie>. [Data uzyskania dostępu: 9 luty 2025].
- [2] „Dźwięk – Wikipedia, wolna encyklopedia,” 3 Lipiec 2023. [Online]. Available: <https://pl.wikipedia.org/wiki/D%C5%BAwi%C4%99k>. [Data uzyskania dostępu: 20 Grudzień 2024].
- [3] „Analiza harmoniczna – Wikipedia, wolna encyklopedia,” 27 września 2024. [Online]. Available: https://pl.wikipedia.org/wiki/Analiza_harmoniczna. [Data uzyskania dostępu: 31 grudnia 2024].
- [4] „Szereg harmoniczny (muzyka) – Wikipedia, wolna encyklopedia,” 6 kwietnia 2024. [Online]. Available: [https://pl.wikipedia.org/wiki/Szereg_harmoniczny_\(muzyka\)](https://pl.wikipedia.org/wiki/Szereg_harmoniczny_(muzyka)). [Data uzyskania dostępu: 31 grudnia 2024].
- [5] „Harmoniczna – Wikipedia, wolna encyklopedia,” 1 września 2021. [Online]. Available: <https://pl.wikipedia.org/wiki/Harmoniczna>. [Data uzyskania dostępu: 31 grudnia 2024].
- [6] „Szereg Fouriera – Wikipedia, wolna encyklopedia,” 11 październik 2024. [Online]. Available: https://pl.wikipedia.org/wiki/Szereg_Fouriera. [Data uzyskania dostępu: 8 luty 2025].
- [7] „Próbkowanie – Wikipedia, wolna encyklopedia,” 11 Lipiec 2024. [Online]. Available: <https://pl.wikipedia.org/wiki/Pr%C3%B3bkowanie>. [Data uzyskania dostępu: 22 Grudzień 2024].
- [8] „Częstotliwość próbkowania – Wikipedia, wolna encyklopedia,” 12 Październik 2024. [Online]. Available: https://pl.wikipedia.org/wiki/Cz%C4%99stotliwo%C5%9B%C4%87_pr%C3%B3bkowania. [Data uzyskania dostępu: 22 Grudzień 2024].
- [9] Z. Rudnicki, „Przetwarzanie analogowo-cyfrowe,” listopad 2016. [Online]. Available: https://home.agh.edu.pl/~zbrudnic/Automatyka/Cw1/Przetw_AC.html. [Data uzyskania dostępu: 8 luty 2025].
- [10] „Kwantyzacja (technika) – Wikipedia, wolna encyklopedia,” 3 Styczeń 2024. [Online]. Available: [https://pl.wikipedia.org/wiki/Kwantyzacja_\(technika\)](https://pl.wikipedia.org/wiki/Kwantyzacja_(technika)). [Data uzyskania dostępu: 22 Grudzień 2024].
- [11] „Sygnał analogowy – Wikipedia, wolna encyklopedia,” 10 styczeń 2023. [Online]. Available: https://pl.wikipedia.org/wiki/Sygna%C5%82_analogowy. [Data uzyskania dostępu: 8 luty 2025].
- [12] „Aliasing (przetwarzanie sygnałów) – Wikipedia, wolna encyklopedia,” 9 Grudzień 2024. [Online]. Available: [https://pl.wikipedia.org/wiki/Aliasing_\(przetwarzanie_sygna%C5%82%C3%B3w\)](https://pl.wikipedia.org/wiki/Aliasing_(przetwarzanie_sygna%C5%82%C3%B3w)). [Data uzyskania dostępu: 22 Grudzień 2024].
- [13] „Szum kwantyzacji – Wikipedia, wolna encyklopedia,” 21 Kwiecień 2023. [Online]. Available: https://pl.wikipedia.org/wiki/Szum_kwantyzacji. [Data uzyskania dostępu: 22 Grudzień 2024].
- [14] „Synteza dźwięku – Wikipedia, wolna encyklopedia,” 16 czerwiec 2023. [Online]. Available: https://pl.wikipedia.org/wiki/Synteza_d%C5%BAwi%C4%99ku. [Data uzyskania dostępu: 7 styczeń 2025].

- [15] V. V. I. k. M. K. Tero Rolonen, „Evaluation of Modern Sound Synthesis Methods,” marzec 1998. [Online]. Available: http://legacy.spa.aalto.fi/publications/reports/sound_synth_report.pdf. [Data uzyskania dostępu: 7 luty 2025].
- [16] S. Bilbao, „Numerical Sound Synthesis,” 27 listopad 2007. [Online]. Available: <https://ccrma.stanford.edu/~bilbao/nssold/booktoplast>. [Data uzyskania dostępu: 7 luty 2025].
- [17] „Dźwięk (muzyka) – Wikipedia, wolna encyklopedia,” 11 styczeń 2025. [Online]. Available: [https://pl.wikipedia.org/wiki/D%C5%BAwi%C4%99k_\(muzyka\)](https://pl.wikipedia.org/wiki/D%C5%BAwi%C4%99k_(muzyka)). [Data uzyskania dostępu: 5 luty 2025].